



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

MASTERS IN ARTIFICIAL INTELLIGENCE AND
DATA ENGINEERING

*Edge AI Computing for Grape Leaf Disease Detection in
LoRaWAN-Based Vineyard Monitoring System*

Advisories:

Prof. Vallati Carlo

Dr. Palla Fabrizio

Prof. Zinnai Angela

Candidate:

Tsegay Teklay Gebrelibanos
ID Number: 683925

Academic Year 2025/2026

Abstract

Existing vineyard monitoring systems primarily rely on environmental data collection such as temperature, humidity, and pressure, transmitted over LoRaWAN networks and providing limited insights into the actual grapevine's health status. This thesis work focuses on enhancing the vineyard monitoring intelligence by integrating Edge AI computing at sensor level for grape leaf disease detection. Moving beyond the traditional environmental-only monitoring approaches, this research was conducted on developing and optimizing deep learning models (CNN for classification and YOLO for detection) and deploying directly on resource-constrained edge sensors. This extension enabled the monitoring system to capture and process leaf images to detect diseases directly at the edge, providing real-time disease detection capabilities while adapting the inherent bandwidth and power limitations of LoRaWAN-based wireless sensor networks. This disease detection feature is implemented using four-stage pipeline architecture and deployed on ESP32-S3-microcontroller. **Stage 1** captures image of vine-trees using the OV3660 camera module operating at VGA resolution with JPEG compression. **Stage 2** employs an ESP-optimized YOLO11n model (ESPDet-Pico: 478KB INT8) to localize individual grape leaves within the captured frame, achieving 56.4% mAP@50 for leaf detection. Then these detected leaf regions are cropped and forwarded to **Stage 3**, where a MobileNetV2 classifier model (2.27 MB INT8) is utilized to identify disease symptoms across the four classes—Black Rot, Esca, Healthy, and Leaf-Blight with $99.8\% \pm 0.065\%$ accuracy on individual leaf patches. **Stage 4** aggregates the classification results from multiple detected leaves to produce a single output formatted for LoRaWAN transmission. Since each captured frame typically contains multiple grape leaves, the weighted aggregation algorithm combines detection confidence, classification entropy, and spatial quality metrics from up to 15 detected leaves to produce robust per-frame disease diagnoses result. The aggregated classification result is then packaged into LoRaWAN uplink packet (e.g. "Leaf Blight, 80% confidence") and transmitted over the WSNs.

To optimize also the power consumption, the system works on duty-cycle operation to capture and preprocess image and apply inference at defined intervals and then enters into deep sleep mode. Overall this thesis work demonstrated the feasibility of deploying deep learning models on resources constrained microcontrollers specifically for disease detection in vineyard monitoring system and precision agriculture applications at large.

Keywords: Wireless Sensor Networks, LoRaWAN, Precision Agriculture, Grape Leaf Disease Detection, Edge AI Computing, Object Detection Model, Image Classification, Quantization, ESP32-S3.



Acknowledgments

I would like to express my sincere gratitude to my advisor, Prof. Carlo Vallati, for his continuous guidance, and insightful feedbacks throughout this thesis work.

I am deeply grateful to the dedicated team at INFN (Istituto Nazionale di Fisica Nucleare) and the Department of Agriculture, University of Pisa for providing me with the opportunity to work on this challenging and rewarding project. Special thanks to my supervisor Dr. Fabrizio Palla and Prof. Angelo Zinnai and Bruno Casu for their technical expertise and mentorship during the development and implementation phases.

I would also like to thank you to the University of Pisa International Office for awarding me the scholarship that helped me to pursue my studies.



Contents

Abstract	i
Acknowledgments	ii
List of Figures	vii
List of Tables	viii
List of Acronyms	ix
1 Introduction and Background	1
2 Literature Review	4
2.1 IoT & LoRaWAN Vineyard Monitoring Systems	4
2.2 Deep Learning Models for Grape Leaf Disease Detection	4
2.3 Lightweight and Edge-Based Grape Leaf Disease Detection Models	7
2.4 Research Gaps on the Existing LoRaWAN-based Vineyard Monitoring System	8
3 System Architecture and Model Design	10
3.1 System Overview and Methodology	10
3.1.1 Why Two Distinct Neural Network Architectures?	10
3.1.2 Four-Stage Pipeline Architecture	11
3.2 Dataset Preparation and Preprocessing	11
3.2.1 The Grape-Leaf Disease Dataset	12
3.2.2 Grapevine Dataset for Leaf Localization	13
3.3 Leaf Localization ESPDet-Pico Model (YOLO-based)	16
3.3.1 Object Detection Framework Selection	16
3.3.2 YOLO11-Optimization for Resources Constrained Deployment	16
3.3.2.1 Backbone Architecture Modifications	17
3.3.2.2 Detection Head Architecture	18



3.3.2.3	Parameter and Memory Reduction	19
3.3.3	ESPDet-Pico Model Training and Optimization	20
3.3.3.1	Input Resolution and Rectangular Training	20
3.3.3.2	Optimization Configuration	20
3.3.3.3	Loss Function	21
3.3.3.4	Data Augmentation Strategy	21
3.3.3.5	Training Convergence	21
3.3.4	Detection Performance Evaluation	22
3.3.4.1	ESPDet-Pico Performance Metrics	22
3.3.5	Post-Training Quantization	22
3.3.6	Comparative Analysis: ESPDet-Pico vs YOLO11n	22
3.3.7	Deployment Feasibility and Trade-off Justification	24
3.4	Disease Classification Model (MobileNetV2)	24
3.4.1	Convolutional Neural Network Architecture	24
3.4.2	Transfer Learning Strategy	26
3.4.2.1	Stage 1: Classifier Head Training	26
3.4.2.2	Stage 2: Full Model Fine-tuning	26
3.4.3	MobileNetV2 Training and Optimization Techniques	27
3.4.4	Model Evaluation and Performance Metrics	28
3.4.5	Post-Training Quantization	29
3.4.6	MobileNetV2 vs ResNet18 Comparative Analysis	29
3.4.6.1	Accuracy-Efficiency Trade-off Analysis	29
3.4.6.2	Statistical Significance of Accuracy Difference	30
3.4.6.3	Architectural Implications for Edge Deployment	30
3.4.6.4	Model Selection Rationale	30
4	Edge AI Deployment on ESP32-S3 Microcontrollers	32
4.1	Target Hardware Platform and Development Framework	32
4.1.1	ESP32-S3-WROOM-1 Microcontroller Architecture	32
4.1.2	ESP-IDF Development Framework	33



4.1.3	ESP-DL Inference Engine	33
4.2	Model Quantization and Format Conversion	34
4.2.1	The ESP-PPQ Quantization Framework	34
4.2.2	ESPDet-Pico Export and Quantization Pipeline	35
4.2.2.1	ONNX Export	35
4.2.2.2	INT8 Quantization	35
4.2.3	MobileNetV2 Export and Quantization Pipeline	36
4.2.3.1	ONNX Export and Simplification	36
4.2.3.2	Calibration Dataset Preparation	36
4.2.3.3	Quantization and ESP-DL Format Conversion	37
4.2.4	Quantization Accuracy Preservation	37
4.3	Firmware Architecture and Memory Management	38
4.3.1	Model Storage Strategy	38
4.3.2	Initialization Order and PSRAM Fragmentation Avoidance	39
4.3.3	Buffer Allocation and Zero-Copy Reuse Strategy	39
4.4	Inference Pipeline Implementation	40
4.4.1	Camera Acquisition and JPEG Decoding	40
4.4.2	Leaf Detection and Coordinate Mapping	41
4.4.3	Region Cropping and Classification Preprocessing	41
4.4.4	Disease Classification and INT8 Dequantization	42
4.4.5	Multi-Leaf Aggregation Strategies	42
4.5	Implementation Constraints and Design Trade-offs	43
4.5.1	Build System and Model Embedding Constraints	43
4.5.2	Class Index Mapping and Framework Compatibility	44
5	Performance Evaluation and Discussion	45
5.1	Inference Latency Analysis	45
5.1.1	Per-Stage Latency Breakdown	45
5.1.2	End-to-End Pipeline Latency and Throughput	46
5.1.3	SIMD Utilization and Computational Efficiency	46



5.2	Model Accuracy Validation	47
5.2.1	ESPDet-Pico Detection Accuracy	47
5.2.2	MobileNetV2 Classification Accuracy	47
5.2.3	Aggregation Strategy Evaluation	48
5.3	Memory Utilization and Runtime Stability	48
5.3.1	Memory Footprint Analysis	48
5.3.2	Long-Term Stability Validation	49
5.4	Power Consumption and Energy Efficiency	50
5.4.1	Active and Sleep Power Measurements	50
5.4.2	Duty-Cycle Energy Analysis	50
5.5	Comparative Analysis with Alternative Edge Platforms	51
5.6	Discussion	52
5.6.1	Trade-offs Between Accuracy, Latency, and Power	52
5.6.2	Limitations and Deployment Considerations	52
6	Conclusion and Future Work	54
6.1	Conclusion	54
6.2	Future Work	55
	Bibliography	57

List of Figures

1.1	LoRaWAN-based vineyard monitoring system architecture that only relies on environmental data for disease risk assessment	2
2.1	Image feature extraction to train IKNN model to detect grape leaf disease [1].	5
2.2	An Edge-boosted Crop Disease Detection System via TinyML and LoRa Communication [2].	8
3.1	System architecture of the proposed edge AI-based grape leaf disease detection integrated into Existing LoRaWAN-based vineyard monitoring. .	10
3.2	Four-stage edge AI pipeline for grape leaf disease detection.	11
3.3	Sample images from the grape leaf disease dataset, illustrating the four classes: Black Rot, Esca, Healthy, and Leaf Blight.	12
3.4	Annotated image from the leaf localization dataset, bounding boxes around individual grape leaves	14
3.5	Training curves for ESPDet-Pico on the grapevine dataset 3.2	21
3.6	ESPDet-Pico detection performance curves on the grape leaf test set. . .	23
3.7	Training and validation curves for the MobileNetV2 model.	28

List of Tables

3.1	Grape leaf disease dataset distribution across training, validation, test splits	12
3.2	Detection training augmentation configuration for ESPDet-Pico	15
3.3	ESPDet-Pico backbone architecture after compound scaling ($d=0.50$, $w=0.25$, $\text{max_ch}=512$)	17
3.4	Comparative evaluation of ESPDet-Pico and YOLO11n baseline on grape leaf detection	23
3.5	Comparative analysis of MobileNetV2 and ResNet18	29
5.1	Per-stage latency breakdown for the inference pipeline (10 detected leaves)	45
5.2	End-to-end latency and throughput versus number of detected leaves . .	46
5.3	ESPDet-Pico detection accuracy: FP32 vs INT8 quantized	47
5.4	MobileNetV2 classification accuracy: FP32 vs INT8 quantized	48
5.5	Sample aggregation output: hybrid spatial-quality weighting (10 leaves) .	48
5.6	PSRAM allocation breakdown during system initialization	49
5.7	Comparison of edge AI platforms for vineyard disease detection deployment	51

List of Acronyms

Acronym	Definition
AI	Artificial Intelligence
CNN	Convolutional Neural Network
DL	Deep Learning
DMA	Direct Memory Access
ESP	Espressif Systems Processor
FP	False Positive
FP32	32-bit Floating Point
GPU	Graphics Processing Unit
INT8	8-bit Integer Quantization
IoT	Internet of Things
LoRaWAN	Long Range Wide Area Network
LPWAN	Low Power Wide Area Network
mAP	mean Average Precision
MCU	Microcontroller Unit
ML	Machine Learning
NN	Neural Network
ONNX	Open Neural Network Exchange
OTA	Over-The-Air
PTQ	Post-Training Quantization
PSRAM	Pseudo-Static Random Access Memory
RAM	Random Access Memory
ROI	Region of Interest
SRAM	Static Random Access Memory
TFLite	TensorFlow Lite
UART	Universal Asynchronous Receiver-Transmitter
WSN	Wireless Sensor Network
YOLO	You Only Look Once

Chapter 1

Introduction and Background

Cultivation of grapes is one of the most economically significant crops in the world, supporting wine production, juice, and fresh fruit markets [3]. However, as global cultivation expands, disease issues increasingly threaten yield and quality, destabilizing the industry [4, 3]. Grapevines are vulnerable to fungal and bacterial diseases including Black Rot, Esca, Leaf Blight, and others that can affect the entire harvest if not detected and managed promptly [3, 5, 6, 7]. Traditional disease identification relies on manual inspection by experts, which is laborious and vulnerable to error through human oversight [8]. In response to these limitations, The transition toward precision agriculture which is characterized by data driven management practices that has emerged as a critical strategy shift to address these challenges [9, 10]. Within this context, the development of intelligent monitoring systems capable of providing real time insights of plant health status become increasingly vital for vineyard management.

The existing vineyard monitoring system was developed as a long-range, low-power wireless sensor network (WSN) platform designed for environmental monitoring and management of vineyards, aiming to provide methods to evaluate and improve the energy consumption of sensor nodes. The goal of this monitoring system was to provide an efficient and reliable way to promptly detect conditions adverse to grapes by measuring environmental data such as temperature, pressure, soil moisture, and humidity levels of vineyard fields. As you saw in figure 1.1 the platform is composed of a set of data acquisition sensors scattered over large open field areas, a gateway node, and a remote server to process and store the acquired data. This monitoring platform is based on the LoRaWAN specification, which is suitable for long-range and low-power lossy networks (LLNs) applications and provides a reliable link layer protocol for the deployed end nodes (sensor nodes) to access and transmit data to a central gateway [11, 12]. The gateway node provides IP connectivity, forwarding the frames from the wireless sensor network to a local application server, which then sends the data to the LoRaWAN network server called The Things Stack. The telemetry data at the servers is gathered, stored in a MongoDB database, and displayed on a Grafana dashboard back to users, providing flexibility in the use and access of measured environmental conditions.

Although the existing LoRaWAN-based vineyard monitoring system showed high capability for long term data collection across large field areas using low power communication, a fundamental limitation is still existing in this current implementations [13, 9]. As some studies noted, systems with primarily rely on environmental data collection such as temperature, humidity, and pressure, transmitted over LoRaWAN networks and providing limited insights into the actual grapevine's health status [14]. Only environmental sensing provides indicators of disease risk rather than direct assessment of plant health status. This gap of LoRaWAN-based vineyard monitoring system, not visualizing the actual grape leaf status, represented as a significant barrier to achieve more intelligent and responsive vineyard management systems.

Meanwhile, deep learning models for plant diseases identification have showed remarkable

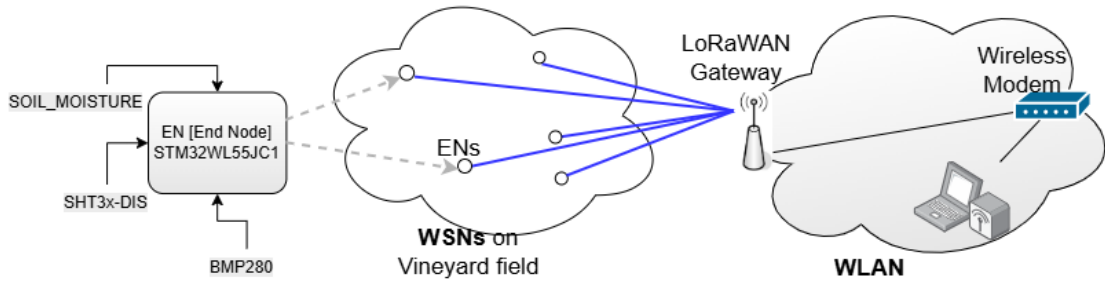


Figure 1.1: LoRaWAN-based vineyard monitoring system architecture that only relies on environmental data for disease risk assessment

performance in laboratory and controlled environments. This proved by a comprehensive review of deep learning for grape leaf disease detection [10]. This review shows convolutional neural networks and transformer based architectures can achieve high accuracy on detecting grape leaf diseases [15, 5]. Object detection frameworks such as YOLO also went further enabling automated leaf localization and disease symptoms in a complex field scenes [16, 17]. Despite these advancements, the practice of deploying these models in vineyard monitoring systems remained limited due two critical challenges. First, deep learning models typically demands high computational resources such as memory, processor and power, which is beyond the capabilities of the edge microcontroller [18]. Second, LoRaWAN network also have payload size constrains. So transmitting raw image data is infeasible.

Edge computing and lightweight deep learning (TinyML) have become promising solutions to address computational resource and bandwidth limitations in LoRaWAN-based agricultural IoT systems. By performing image processing and inference at the sensor level, edge AI reduces the need for high-bandwidth data transmission. This lightweight vision models for disease detection in such resource-limited settings requires careful balance between model capacity and computational constraints [19, 20, 18]. However, most of the edge-based plant disease detection systems have been implemented on single-board computers such as Raspberry Pi or NVIDIA Jetson or smartphones, which offers much more computational resources than microcontrollers [21, 22, 2, 23]. Therefore, such kinds of edge computing are unsuitable for vineyard monitoring system which requires the edge computing for long term and battery powered deployment.

To address this gap and to further enhance the intelligence of the existing LoRaWAN-based vineyard monitoring system, this research introduced an edge AI extension feature focused on edge computing for grape leaf disease detection. The proposed extension Integrated edge AI at sensor level, by moving beyond the traditional environmental monitoring, for nearly real-time visual-based insights into grapevines. By processing leaf images and running CNN classification model at the edge sensor, the system improves its ability to identify grape diseases. This differentiates the monitoring system from other systems that only rely on environmental data, thereby providing a more intelligent and efficient solution for vineyard management. This visual-based disease detection required tactical use of Edge AI computing to overcome the bandwidth limitations inherent in LoRaWAN communication for transmitting image data. End-to-end inference pipeline was deployed on ESP32-S3 microcontroller, which is a low-power microcontroller with AI ca-

pabilities, enabling image acquisition, leaf localization, and disease classification directly at edge. The system transmit only compacted inference results e.g. “Leaf Blight” with confidence 90%, which is then seamlessly appended to the existing LoRaWAN payload. Besides, Duty-cycled operation was implemented to ensure effective power consumption, by setting specified periodic intervals for image capture and on-device processing, followed by extended sleep periods. Therefore, it aligns perfectly with a low-power, event-driven monitoring strategy, providing actionable insights while minimizing the impact on battery lifetime. Moreover, this comprehensive approach enhances the vineyard monitoring system by enabling early, localized detection of plant health issues.

The main aim of this research was to design, implement, and validate Edge AI system for grape leaf disease detection deployed on microcontroller within the existing LoRaWAN-based vineyard monitoring system. Specifically, this research introduces the direct visual-based disease detection into the existing environmental only monitoring system, by deploying a complete inference pipeline on ESP32-S3 microcontroller. Then the system integrates image acquisition, leaf localization within complex vineyard scenes, Multi-class disease classification, and inference results formatted to fit within LoRaWAN payload. Hence, contribution of this study have been diverse, addressing both technical and practical challenges of edge AI deployment for precision agriculture. It demonstrated feasibility of deploying these two-stage, object detection then classification pipeline on ESP32 microcontroller under strict computing resources constraints. This is a significant advancement beyond existing implementations that rely on powerful single-board computers or smartphone devices [21, 22, 23]. Also introduces a weighted aggregation on the classification results from multiple leaves within a single frame, to send a compacted and robust results over LoRaWAN communication.

Overall, this work provide a complete system architecture that filled the gap between environmental only monitoring and direct visual based disease detection. This research established a foundation for Intelligent precision agricultural systems that combine the reliability of LoRaWAN with optimized deep learning models deployed at the edge.

The thesis is organized in six chapters. Chapter 1 introduces the background of the research, outlining the research gap, main aim, and contributions of the study. Chapter 2 reviews related works in the fields of IoT and LoRaWAN for precision agriculture, deep learning for grape disease detection, and edge computing for plant disease detection. Chapter 3 describes the proposed system architecture and model design, including dataset preparation, the ESPDet-Pico leaf localization model, MobileNetV2 disease classification model, and multi-leaf aggregation strategy. Chapter 4 presents the edge AI deployment pipeline on the ESP32-S3 microcontroller, covering model quantization, firmware architecture, memory management, and inference pipeline implementation. Chapter 5 reports the performance evaluation and discussion, including latency analysis, accuracy validation, memory utilization, power consumption, and comparative analysis with alternative platforms. Finally, Chapter 6 concludes the thesis by summarizing the findings, discussing implications, and outlining directions for future research.

Chapter 2

Literature Review

The literature review provides a comprehensive survey of relevant research in wireless sensor networks for agriculture, plant disease detection using computer vision, low-power communication protocols, and edge AI deployment on resource-constrained platforms. The review is organized thematically to highlight how prior work motivates the contributions of this thesis and to identify gaps that Edge AI-enabled visual monitoring addresses.

2.1 IoT & LoRaWAN Vineyard Monitoring Systems

The traditional vineyard monitoring systems are focused on environmental data acquisition through wireless sensor networks. Hnatiuc et al. developed an intelligent grapevine disease detection using IoT sensor network that integrates environmental sensors for monitoring temperature, humidity, and atmospheric conditions [9]. This approach represents the foundational methodology where IoT sensors collect environmental data to predict plant health status. Next, Gawande et al. proposed early prediction of grape disease occurrence using a hybrid classifier model in association with IoT sensors, demonstrating the integration of machine learning with IoT for environmental monitoring [13]. Their work emphasized that monitoring environmental parameters via wireless sensor networks enables the prediction of plant diseases [24].

The adoption of LoRaWAN technology represents a significant advancement for large-scale vineyard monitoring due to its nature in long-range, low-power characteristics. Kirubakaran and Samreen proposed a Grape Leaf Diseases Identification System Using Convolutional Neural Networks and LoRa Technology, which is an early attempt to combine the CNN diseases detection with LoRa network communication [25]. More recently, Hnatiuc et al. made a comprehensive study on IoT monitoring for early identification of diseases in grapevines, integrating AI with IoT sensor networks [14]. Their study revealed how the integration of AI with IoT sensor networks can enhance the diseases' detection in grapevines, but it was primarily focused on environmental data collection and did not explore the edge-based visual-based monitoring.

2.2 Deep Learning Models for Grape Leaf Disease Detection

The usage of deep learning techniques in grape leaf disease detection brought some improvement, evolving from the traditional machine learning approaches into sophisticated Neural Network architectures. Aher et al. provided comprehensive review on deep learning for grape leaf disease detection [10], by putting similar methods together and identifying key challenges in that domain. The early approaches were employed using the classical machine learning techniques, as demonstrated by Shantkumari and Uma, who used machine learning techniques for detection of grape leaf disease using SVM and IKNN [1]. Javidan et al. further explored diagnosis of grape leaf diseases using automatic K-means clustering and machine learning [26], achieved reasonable accuracy but with extensive

manual feature engineering.

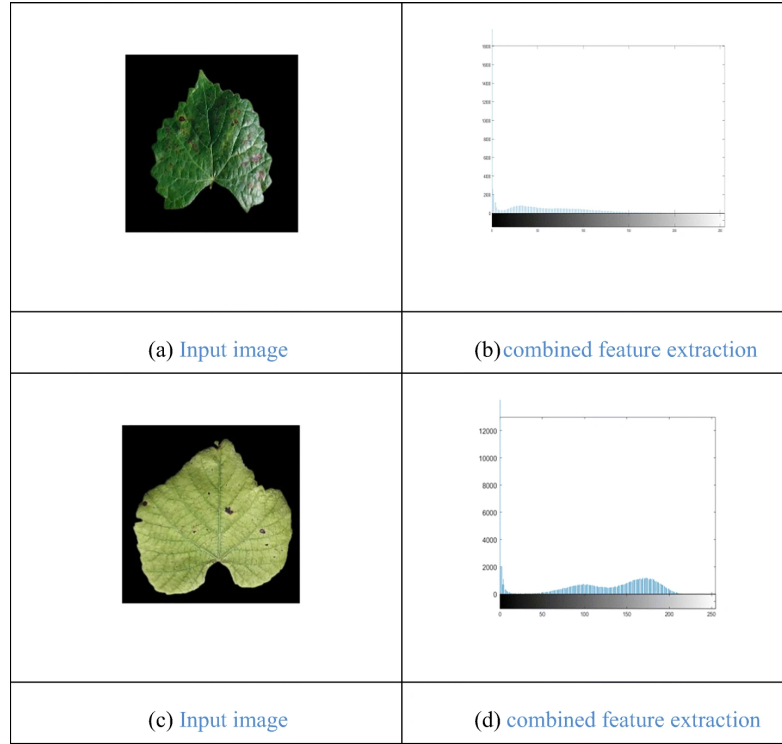


Figure 2.1: Image feature extraction to train IKNN model to detect grape leaf disease [1].

The transition to CNN showed a significant change in detection precision and automation. Diana Andrushia et al. introduced image-based disease classification on grape leaves using convolutional capsule network architectures[27], achieving accuracy of 99.12%. Nargotra and Sharma demonstrated grape leaf disease detection using deep learning with CNN-based approaches[28], while Soni explored deep learning for accurate grape disease classification in ResNet50 architecture[29]. The evolution towards the sophisticated architectures is evidence on Praveen et al.'s work on the novel classification approach for grape leaf disease detection based on different attention deep learning techniques[30], which applied attention mechanism to enhance feature descriptions.

Since the transfer learning became the dominant strategy to deal with not enough training data problems. Vo, Kheo and Nhon et al. together they proposed fusing Xception and ResNet50 features for robust grape leaf disease classification[31], demonstrating that combining several pre-trained architectures yields higher performance. Yadav et al. used the using VGG16 & Inception V3 models to investigate enhanced disease detection prediction accuracy of grape leaves[32], comparing deep learning architectures for classification tasks. Khursheed and Kaur presented improved grape leaf disease detection with GAN-augmented ResNet and EfficientNet [33], utilizing generative adversarial networks for data augmentation to address dataset imbalance.

The recent advancement have been focused on transformer-based architectures and hybrid ones. Zhang et al. developed a grape leaf disease recognition using the transformers [34], shows how capable transformer designs are architectures for disease recognition. Li et al. also came with the grape disease detection system using transformer's vision and environmental sensing[35], representing a novel approach that fuses visual and environ-

mental data streams. Karthik et al. suggested the idea of GrapeLeafNet which is a dual-track feature fusion network using inception-ResNet and shuffle-transformer for accurate grape leaf disease identification[15], achieving high accuracy scores because of its dual-track feature combination. Another further contribution done by Zhang et al. the cross-resolution transformer for grape leaf diseases segmentaion using context mining[36], demonstrating that cross-resolution transformer architecture design enables the precise disease segmentation.

There are some special architectures made for specific detection challenges. Talaat et al. Introduced a DeepLeaf, an optimized deep learning approach for automated recognition of grapevine leaf diseases[5], which optimize neural architectures for multi-class disease classification. Combining deep learning with manually created features, Atesoglu and Bingol suggested the detection and classification of grape leaf diseases with enhanced hybrid model based on feature engineering[7]. Combining deep feature extraction with traditional classifiers in hybrid techniques can produce competitive performance, as demonestrated by Ishengoma and Lyimo's ensemble model for grape leaf disease detection using CNN feature extractors and random forest classifier[37]. Using grape leaf disease identification with sparse data via GAN and CNN, Chen and Wu addressed data scarcity by means of GAN-based data augmentation to achieve the precision agriculture metrics[38].

The integrating object detection frameworks have made it possible to simultaneously localize and classify plant diseases in complex field environments. The YOLOv7 was used by Yang et al. to study the detection of small lesions on grape leaves [16], and he addressed the difficulty spotting the early-stage disease symptoms. Through architectural modifications, Chen et al. developed an enhanced lightweight grape leaf disease detection method [17], therefore improving it performance. Ji et al. carried out a research on grape leaf disease detection method based on NMA-YOLOv8n by introducing a novel attention mechanisms to YOLO architectures[39]. Particularly in relation to boundary detection issues in the situations of dense foliage, Zhou and Yu suggested a lightweight grape leaf disease identification for complex field scenes-YOLO-Grape model and its boundary-lesion coalesce optimization policies[6].

The next focus has been given to domain-specific challenges including multi-disease detection and disease severity evaluation. Mehta et al. investigated the grape leaf disease severity analysis using federated learning with CNN techniques[40], hence bringing privacy-preserving distributed learning for disease assessment. Combining spatial and temporal data for dynamic disease monitoring, Gaur et al. worked on fighting grape black rot with deep learning using CNN-LSTM hybrid model for disease severity classification[41]. In a review of advancements in deep learning for accurate classification of grape leaves and diagnosis of grape diseases[42], Kunduracioglu and Pacal chronicled the change from binary classification to multi-class, multi-disease detection systems. Many other contributions include work by Naik et al. on disease detection in grape leaves[43], Shetty and Shetty on grape leaf disease detection using deep learning approach"[44], and Liu and Li on grape leaf disease classification based on convolutional neural networks"[4]. These all contributions represent the ongoing evolution of deep learning techniques for grape leaf disease detection, with a clear trend towards more sophisticated architectures, multi-disease detection capabilities and consideration of real-world complexities.

2.3 Lightweight and Edge-Based Grape Leaf Disease Detection Models

The practical deployment of deep learning models for detecting grape leaf diseases in vineyard fields requires considering the constraints of available computational resources. Even though they achieved high performance in detecting grape leaf diseases, the deployment of these models on resource-constrained edge devices remains challenging. Hence, some researchers focused on developing lightweight neural network architectures specifically designed for edge deployment.

The deployment of deep learning models on the resource-constrained edge devices, at sensor level, is crucial step towards practical, real-time agricultural monitoring systems. Lin et al. started this research direction by developing a model named GrapeNet, a lightweight convolutional neural network model for identification of grape leaf diseases [45]. Their work demonstrated that optimized design of lightweight architectures while maintaining acceptable accuracy and significantly reducing computational requirements. This study established a foundation for "the lightweight convolutional neural networks" as a key design principle for edge deployment in resources-constrained applications.

Subsequently, research started to focus neural networks architectures designed and optimized for edge computing. Particularly MobileNet family, due to their depthwise separable convolutions and efficiently designed architectures to reduce computational complexity and make them suitable for resource-constrained devices. Sahid and Cahyadi developed image classification using MobileNet architecture for grape leaf disease detection[46], in a reduced parameter count and computational complexity. Meanwhile, Mathew et al. explored a real-time detection of grape leaf diseases with MobileNetV2 deep learning network[3], achieving real-time inference capabilities on edge hardware. Further improvements introduced attention mechanisms to MobileNetV2 architecture, while maintaining its lightweight characteristics, as shown by Dharrao et al.[8]. Hybrid approach, combining MobileNetV2 with transformer architectures have also been explored to increase performance while keeping the model lightweight[47].

Specialized lightweight architectures have been explicitly designed for resource-constrained edge device deployment. This research direction is initiated by Deepa et al. who introduced L-Net, lightweight CNN framework for sustainable multicrop leaf disease detection and classification on edge devices[48]. Their work highlighted importance lightweight architectural designs beyond MobileNet. Parallely, Zhang et al. explored a lightweight grape leaf disease recognition method based on transformer frameworks[34], demonstrating that even transformer architectures can also effectively compress for edge deployment without significant reduction in accuracy.

Since these works showed the effectiveness of lightweight designs, the integration of edge computing with practical deployment platforms has gained significant momentum. Tao et al. developed iCrop+, an edge-boosted crop disease detection system via TinyML and LoRa communication[2], demonstrating the integration of TinyML-based image segmentation on Raspberry Pi with LoRa communication for crop disease monitoring. Their system employs TinyML for edge-based image segmentation, followed by CNN inference on the segmented images, with results transmitted over LoRa communication, as shown

in figure 2.2. This work represents a significant step towards practical disease monitoring by integrating edge inference with low-power wireless communication, though deployment remains on single-board computers rather than microcontroller-class devices. Karim et al. also presented a landmark study on enhancing agriculture through real-time grape leaf disease classification via an edge device with a lightweight CNN architecture and Grad-CAM[21], demonstrating that lightweight CNN architectures can deploy and perform inference on resource-constrained edge devices. proceeding this work, Lazcano-García et al. developed a deep learning-based system for early symptoms recognition of grapevine red blotch and leafroll diseases and its implementation on edge computing devices[22], specifically addressing early disease detection on embedded platforms.

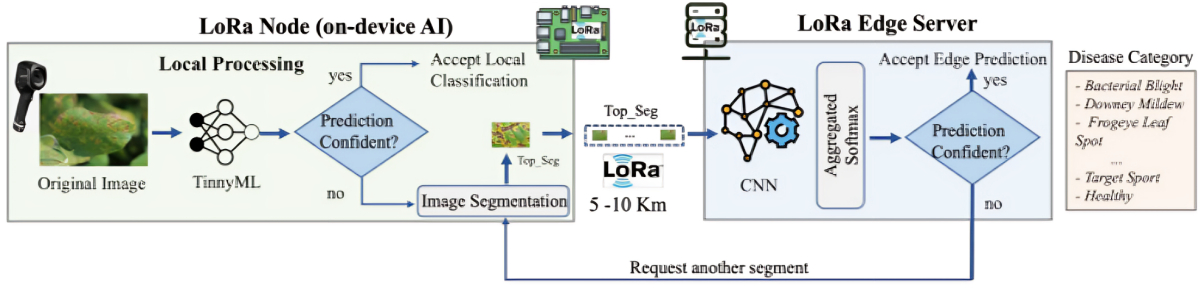


Figure 2.2: An Edge-boosted Crop Disease Detection System via TinyML and LoRa Communication [2].

More recently, some researchers start investigating microcontroller-level deployment, in the most resource-constrained scenario. Grujev et al.[49] demonstrated real-time apple disease classification approach using lightweight neural network on an Arduino edge device. Similarly, Amirkhanova et al.[50] developed a lightweight, end-to-end encrypted data pipeline for IoT. Here they explored ESP32 for secure communication in IoT, but they did not use it for visual based inference.

Another, model compression techniques have become crucial enablers for edge deployment. Quantization and pruning are common techniques to compress models. Joshi examined edge-AI for agriculture using lightweight vision models for disease detection in resource-limited settings[19], emphasizing that Edge AI systems requires careful balance between model accuracy and computational efficiency.

2.4 Research Gaps on the Existing LoRaWAN-based Vineyard Monitoring System

Despite the significant advancement progresses in both WSNs for vineyard monitoring and deep learning for grapevine disease detection, a crucial gap, combining them for practical, real-time disease detection still exists. Existing LoRaWAN-based vineyard monitoring systems, also demonstrated by Hnatiuc et al.[9], Gawande et al.[13], and Almufareh et al. [11], primarily focuses on environmental sensing. While these systems successfully transmit temperature, humidity, and atmospheric data over LoRaWAN networks, and they can provide indirect indicators of disease risk rather than direct visual based disease diagnosis.

Conversely, the substantial body of work on deep learning for grape leaf disease detection[15, 36, 5], they achieved high accuracy under unconstrained computational environments.

However, these models demands large computational resources, making them unsuitable for microcontroller level deployment.

Although lightweight and edge-based disease detection [21, 22, 48], have moved towards the embedded level deployment, like in Raspberry Pi, NVIDIA Jetson, and smartphones. However, these devices still consumes high power, so they cannot fit into the LoRaWAN-based vineyard monitoring system.

Moreover, the integration of complete inference pipelines, including image capturing, object detection, disease detection, and result aggregation on battery powered microcontroller within LoRaWAN networks remains unexplored. The existing efforts are either perform classification on pre-cropped images [25], rely on Raspberry Pi-level hardware [2], or assume smartphone-class resources [23].

Beyond the broader integration challenges, the technical deployment of both leaf localization and disease detection models into a single microcontroller is not investigated yet. Existing YOLO-based approaches for grape leaf detection[16, 17, 39], have focused on detection detecting the disease directly. They did not explore the integration of a two-stage pipeline where a lightweight model performs leaf localization followed by a separate classifier for disease classification. Similarly, Zhou and Yu[6] addressed leaf boundary detection in complex scenes but did not extend their work to additional model integration or wireless communication.

In summary, the core research gap addressed by this thesis is the lack of integrated, edge AI computing systems with capability of performing visual based grape leaf disease detection integrated into the existing LoRaWAN-based vineyard monitoring systems. Thus, this thesis work contributed in addressing this problem by developing Edge AI inference pipeline using ESP32-S3 microcontroller, camera, lightweight YOLO leaf detection model, MobileNetV2 multiclass disease classifier, under LoRaWAN communication constraints. Also duty-cycled operation with deep sleep power management is implemented to ensure long-term deployment feasibility. The following chapter presents the proposed system architecture and model design that addresses these identified gaps.

Chapter 3

System Architecture and Model Design

3.1 System Overview and Methodology

This chapter presents the design and training of the two neural network models at the core of the edge AI-based grape leaf disease detection system, integrated into the existing vineyard monitoring infrastructure as shown in Figure 3.1. It starts with an overview of the system architecture, followed by detailed descriptions of the datasets in Section 3.2. Then, the design and training of the leaf localization model and the disease classification model are presented in Sections 3.3 and 3.4 respectively. The hardware platform description and full deployment implementation are covered in Chapter 4.

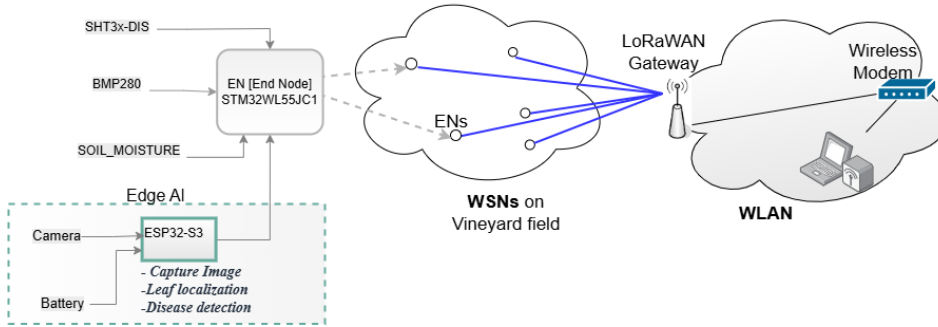


Figure 3.1: System architecture of the proposed edge AI-based grape leaf disease detection integrated into Existing LoRaWAN-based vineyard monitoring.

3.1.1 Why Two Distinct Neural Network Architectures?

The main purpose of the proposed system is to diagnose grapevine diseases by analyzing their leaves. While the training dataset consists of single leaf images labeled with different disease classes. In real-world monitoring scenarios, the input images captured by camera contain multiple leaves in single frame. Therefore, training model with single leaf images to infer full frame is not effective. To address this, the system uses two distinct neural network architectures. One to localize and extract the target leaf regions from the image frame. The other to diagnose the disease symptoms on the extracted leaves.

Additionally, the leaf localization and disease classification tasks have different characteristics requiring specific architectures and training strategies. By separating these tasks into two specialized models, each can be optimized for its specific task. The complete deployment pipeline of integrating these two domain specific models is explained in Section 3.1.2, with detailed model architectures in Sections 3.3 and 3.4.

3.1.2 Four-Stage Pipeline Architecture

The disease detection system is implemented in a four-stage pipeline deployed on ESP32-S3 microcontrollers, as illustrated in Figure 3.2.

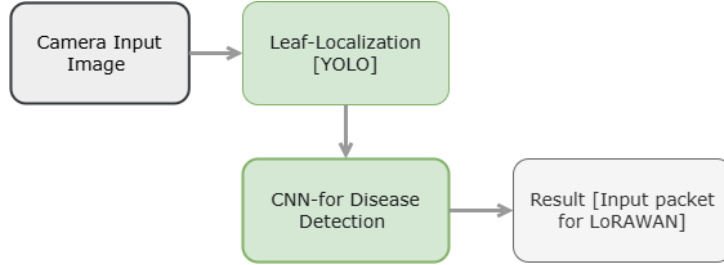


Figure 3.2: Four-stage edge AI pipeline for grape leaf disease detection.

Stage 1: Image Acquisition - An OV3660 camera module capture grapevine images at VGA resolution (640×480). This module is designed to receive parallel video data signals, and its bus supports an 8-bit 16-bit DVP image sensor, with clock frequency of up to 40 MHz[51].

Stage 2: Leaf Localization - A YOLO-based detection model (ESPDet-Pico) localizes individual grape leaves within the captured frame. This stage identifies and crops leaf, the regions of interest. The ESP-optimized model architecture and its training procedure are detailed in Section 3.3.

Stage 3: Disease Classification - The detected leaves at stage 2 are resized and classified by a MobileNetV2 model trained on different classes of grape disease single leaf images. The details of this model training, quantization, and evaluation are presented in Section 3.4.

Stage 4: Result Aggregation - Since a single frame contains multiple leaves, predictions should be aggregated using weighted combination of detection confidence, classification entropy, and spatial quality metrics. The aggregated result is encoded into LoRaWAN payload. The aggregation strategy and its design rationale are detailed in Section 4.4.5.

3.2 Dataset Preparation and Preprocessing

Two distinct datasets were used to train the leaf localization and disease classification models. The disease classification trained on dataset that contains single-leaf images on uniform backgrounds, enabling the model to learn disease-specific visual features without background interference. In contrast, the leaf localization trained on dataset that contains natural scenes of grapevine with annotated leaf bounding boxes, reflecting real deployment conditions characterized by occlusions, illumination variability, and background complexity.

In a real-world monitoring scenario, input images captured by camera contains multiple leaves in single frame with complex backgrounds and other environmental factors. So, during inference, localization model first detects leaf regions from full frame vineyard images. Then, the disease classification model diagnoses the disease symptoms on the

extracted leaf. This task separation allows each model to specialize in its respective tasks while maintaining robustness under real-world field conditions.

3.2.1 The Grape-Leaf Disease Dataset

The grape-leaf disease dataset sourced from the PlantVillage Dataset hosted on Kaggle, created by Abdallah Ali [52]. PlantVillage is a widely adopted public repository of plant disease imagery developed through collaborative science efforts. The grape leaf subset aggregates pathology images captured under controlled laboratory conditions, with consistent lighting and uniform backgrounds. The dataset emphasizes leaf-level disease manifestations across four epidemiologically significant categories affecting the cultivation of grapevines worldwide. The dataset contains 4,062 RGB images distributed across four distinct disease classes: *Black Rot* (caused by *Guignardia bidwellii*), *Esca* (Black Measles, a complex disease involving multiple fungal pathogens), *Leaf Blight* (Isariopsis Leaf Spot, caused by *Pseudocercospora vitis*), and *Healthy* (disease-free grape leaf samples). Table 3.1 presents the complete class distribution statistics and train/validation/test splits. Sample images from each class are shown in Figure 3.3.

Table 3.1: Grape leaf disease dataset distribution across training, validation, test splits

Disease Class	Training	Validation	Test	Total
Black Rot	803	141	236	1,180
Esca (Black Measles)	940	166	277	1,383
Healthy	287	51	85	423
Leaf Blight	731	129	216	1,076
Total	2,761	487	814	4,062

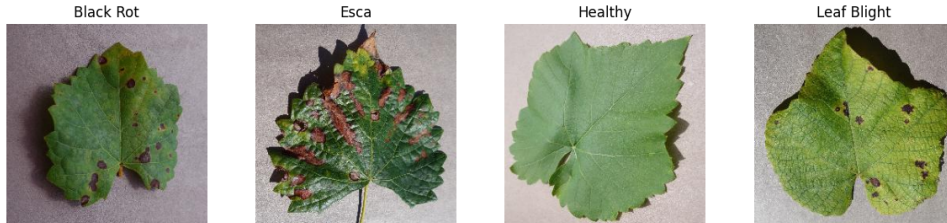


Figure 3.3: Sample images from the grape leaf disease dataset, illustrating the four classes: Black Rot, Esca, Healthy, and Leaf Blight.

The dataset uses a stratified partitioning with fixed 80/20 train-test split, allocating 3,248 images for model training and 814 images for final model testing. The training set is further subdivided into 85% training (2,761 images) and 15% validation (487 images). This split enables hyperparameter tuning on the validation set while preserving an independent test set for unbiased performance assessment at the end. Stratified sampling maintains class level proportion across all partitions, preventing from training bias toward majority classes.

Preprocessing Techniques

The input size for the classification model is set to 128×128 pixels, balancing model capacity with the memory constraints of the ESP32-S3 platform. The original images in

the dataset are 256×256 pixels. To train the model, images are resized to 144×144 pixels and then randomly cropped to 128×128 pixels. But in validation and testing, it crops the center region of the resized image instead of random cropping. This allows the model to learn to focus on the most relevant leaf features while discarding edge pixels that may contain noise or artifacts. The random cropping during training also exposes the model to varied spatial contexts improving generalization.

All images are normalized using standard ImageNet preprocessing statistics (mean $\mu = [0.485, 0.456, 0.406]$ and standard deviation $\sigma = [0.229, 0.224, 0.225]$) as common practice in transfer learning with ImageNet pretrained models. Pixel intensities are first scaled to $[0, 1]$ by dividing by 255, then standardized via $x_{\text{norm}} = (x - \mu)/\sigma$. This normalization aligns input distributions with the pretrained MobileNetV2 network, facilitating effective transfer learning and fine-tuning convergence.

Training Dataset Augmentation

Data augmentation applied stochastically on-the-fly during each training epoch using PyTorch's `torchvision.transforms`, ensuring no two identical augmented samples are generated across epochs. This helps to prevent overfitting when dataset is not large enough to generalize well on unseen data. The augmentation pipeline of training set includes geometric and photometric transformations. Geometric augmentation include random horizontal and vertical flips, rotations, perspective distortions, and occlusions. Photometric augmentation include color jittering, grayscale conversion, and Gaussian blurring. These transformations are applied to simulate real leaf appearance cause by different lighting conditions, camera angles, and environmental backgrounds.

These augmentations applied with specific probabilities and ranges to balance the training dataset diversity while mimicking realistic leaf appearance. However, no augmentation is applied to validation and test sets. This ensures that model performance is evaluated on the original input data.

3.2.2 Grapevine Dataset for Leaf Localization

The leaf localization dataset comprises 531 natural vineyard photographs with manually annotated bounding boxes, enabling training of region-of-interest (ROI) detection models for unconstrained deployment scenarios. Unlike the classification dataset that controlled laboratory conditions, these images capture real-world vineyard complexity. Images with variable leaf densities, overlapping foliage, occluded leaves, diverse lighting conditions, and cluttered backgrounds. The images are sourced from the Grape/Grapes dataset on Kaggle [53], curated by Nicolaas Regnier.

Preprocessing Techniques

On this dataset preprocessing stage annotation, conversion to YOLO format, input resizing, and intensity normalization are applied. Image annotations were done manually with the assistance of the Segment Anything Model in the Roboflow platform. Each visible leaf instance in the vineyard images was centered with a bounding box and labeled as a single class (leaf). The bounding box coordinates were normalized to the YOLO format, which specifies each box via its center position (x_c, y_c) , width w , and height h , all scaled to the range $[0, 1]$ relative to the image dimensions. This normalization allows the

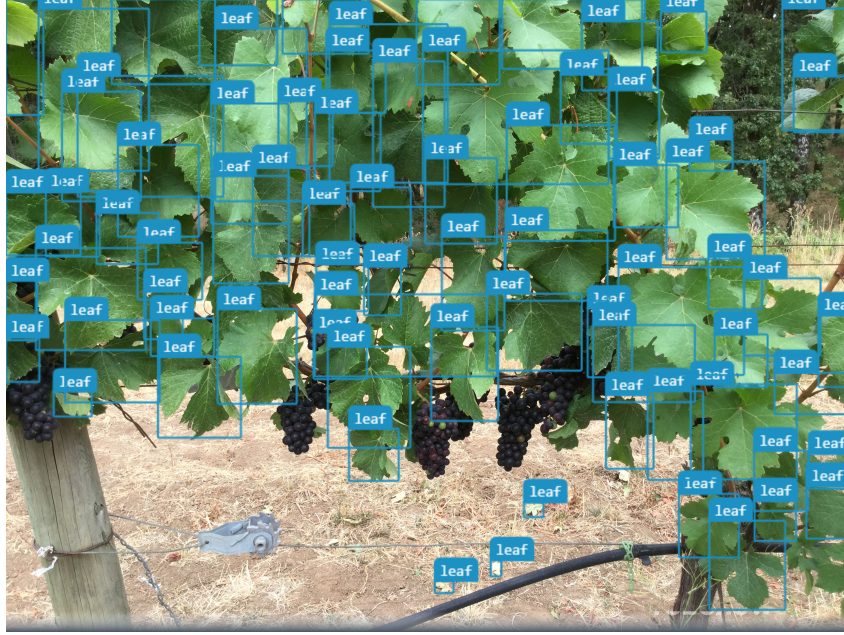


Figure 3.4: Annotated image from the leaf localization dataset, bounding boxes around individual grape leaves

model to learn scale-invariant representations of leaf regions, facilitating training across variable input sizes and resolutions. The complete annotated dataset, titled “Grape Leaf ROI Detection”, is now publicly available on the Roboflow platform [54] and partitioned into training (328 images, 64%), validation (82 images, 16%), and test (103 images, 20%) sets. The partitioning maintains natural distributions of leaf counts and scene complexity across splits, ensuring that the model is trained and evaluated on representative samples of real-world vineyard conditions. Figure 3.4 illustrates a sample annotated image from the dataset, showcasing bounding boxes around individual grape leaves in a complex vineyard scene.

The annotated dataset resizes input images to 416×320 pixels (width $\times$ height) for training the detection model. To fit the model’s input requirements while preserving aspect ratios, rectangular training (`rect=True`) is employed. This approach resizes images via letterboxing, scaling them to fit within the target dimensions while maintaining their original aspect ratio. The remaining space is filled with black padding, preventing distortion artifacts that would otherwise degrade detection accuracy on elongated or irregularly shaped leaves. This preprocessing strategy minimizes computational overhead on the ESP32-S3 device by avoiding complex resizing operations during inference, allowing the model to operate directly on the camera’s native output resolution.

For intensity normalization, YOLOv11 utilizes a linear scaling strategy where raw pixel intensities are mapped to the range $[0, 1]$ by dividing by the maximum 8-bit value: $x_{\text{norm}} = x_{\text{raw}}/255$. Unlike many deep learning architectures that require specific ImageNet-derived mean and standard deviation shifts, YOLO’s approach minimizes preprocessing overhead while remaining strictly consistent with its pre-training regime. This alignment ensures the preservation of learned features to transfer learning for specialized tasks, such as leaf detection.

Training Augmentation Pipeline

Leaf localization requires precise bounding box regression, necessitating a balanced augmentation strategy that enhances generalization without introducing excessive localization noise. Unlike classification, where aggressive color perturbation improves disease recognition, detection training prioritizes geometric robustness and leaf structure preservation. The augmentation pipeline is organized into three categories—photometric, geometric, and advanced compositing—summarized in Table 3.2.

Table 3.2: Detection training augmentation configuration for ESPDet-Pico

Category	Transform	Value
Photometric	Hue shift	± 0.015
	Saturation shift	± 0.4
	Value shift	± 0.4
Geometric	Rotation	$\pm 10^\circ$
	Translation	$\pm 10\%$
	Scale	$0.5\text{--}1.5\times$
	Perspective	0.0001
	Vertical flip	$p = 0.5$
	Horizontal flip	$p = 0.5$
Advanced	Mosaic	$p = 0.5$
	MixUp	$p = 0.05$
	Close mosaic	epoch 50

Photometric augmentations operate in the HSV color space. Hue variation is intentionally minimal (± 0.015) to preserve the characteristic green coloration that distinguishes foliage from background, while moderate saturation and brightness perturbations (± 0.4) account for natural lighting variability in vineyard environments.

Geometric augmentations simulate realistic camera positioning scenarios: rotation and shear approximate mounting angle deviations, translation models slight positional shifts, and scale variation covers camera-to-plant distance changes. Both vertical and horizontal flips are applied with 50% probability. Perspective distortion is kept negligible (0.0001) to avoid unrealistic warping.

The advanced compositing techniques are YOLO-specific augmentations designed for multi-scale robustness. Mosaic augmentation combines four training images into a single grid, improving small-leaf detection and multi-scale feature learning. MixUp probability is intentionally reduced to 5% (versus a typical 15%) because excessive pixel-wise blending degrades bounding box localization precision. CopyPaste augments effective dataset size by transferring leaf instances between images, improving occlusion handling. Mosaic is disabled after epoch 50 (`close_mosaic=50`) out of 1000 total epochs, allowing the remaining 950 epochs to fine-tune on unmodified images that more closely approximate real inference conditions.

Validation and test sets receive no augmentation, undergoing only letterboxing and normalization to ensure that performance metrics reflect true model capability on canonical inputs.

With both datasets prepared and preprocessing pipelines established, the following sections detail the design, training, and optimization of the leaf localization (Section 3.3) and disease classification (Section 3.4) models for embedded ESP32-S3 deployment.

3.3 Leaf Localization ESPDet-Pico Model (YOLO-based)

The first stage of the inference pipeline requires accurate localization and extraction of individual grape leaf regions from unconstrained vineyard imagery. This preprocessing step enables subsequent disease classification by isolating leaves from complex backgrounds, overlapping vegetation, and varying illumination conditions. Given the computational constraints of ESP32-S3 microcontrollers, the leaf localization model must satisfy two competing requirements: sufficient detection accuracy to reliably identify leaf boundaries, and minimal model complexity to enable real-time inference on resource-constrained hardware.

3.3.1 Object Detection Framework Selection

Modern object detection architectures can be broadly categorized into two paradigms: anchor-based two-stage detectors (e.g., Faster R-CNN [55]) and single-stage detectors (e.g., YOLO [56], SSD [57]). Two-stage detectors achieve high accuracy through region proposal networks followed by refinement stages, but incur substantial computational overhead unsuitable for embedded deployment. Single-stage detectors directly predict class probabilities and bounding box coordinates in a single forward pass, offering superior inference speed at moderate accuracy cost—an advantageous trade-off for edge AI applications.

The YOLO (You Only Look Once) family [58] represents the state of the art in efficient single-stage detection. YOLO11n, the nano-scale variant developed by Ultralytics [58], serves as the baseline model for this work. YOLO11n employs an anchor-free detection paradigm, eliminating the need for predefined anchor boxes and simplifying post-processing. The architecture comprises a feature extraction backbone with cross-stage partial connections, a feature pyramid network (FPN) [59] for multi-scale fusion, and detection heads operating at three spatial resolutions to capture objects across varying scales.

YOLO11n achieves strong detection performance with 2.59 million parameters and a model size of 5.23 MB. However, these resource requirements exceed the 8 MB PSRAM capacity of the ESP32-S3 platform when accounting for activation memory, input buffers, and runtime overhead. Direct deployment of YOLO11n would necessitate model streaming from external flash storage, introducing prohibitive latency that precludes real-time operation. Consequently, aggressive model compression strategies are required to reduce memory footprint while preserving detection accuracy.

3.3.2 YOLO11-Optimization for Resources Constrained Deployment

To enable ESP32-S3 deployment, we adopt ESPDet-Pico, an ESP-optimized object detector derived from the YOLO11 architecture. ESPDet-Pico applies systematic architectural pruning, replacing computationally expensive operations with hardware-friendly alternatives while maintaining the core detection pipeline.

3.3.2.1 Backbone Architecture Modifications

The feature extraction backbone comprises 11 layers organized into four downsampling stages, achieving spatial resolutions of P3/8, P4/16, and P5/32 relative to the input image. Table 3.3 details the complete backbone architecture after applying the compound scaling factors ($d = 0.50$, $w = 0.25$).

Table 3.3: ESPDet-Pico backbone architecture after compound scaling ($d=0.50$, $w=0.25$, $\text{max_ch}=512$)

Index	Layer Type	Output Channels	Stride	Output Scale
0	Conv	16	2	P1/2
1	DSCnv	32	2	P2/4
2	ESPBlockLite	64	1	P2/4
3	DSCnv	64	2	P3/8
4	DSC3k2	64	1	P3/8
5	SCDown	64	2	P4/16
6	DSC3k2	64	1	P4/16
7	SCDown	128	2	P5/32
8	DSC3k2	128	1	P5/32
9	SPPF	128	1	P5/32
10	DSCnv [†]	128	1	P5/32

[†]Uses 7×7 kernel (padding 3); all other DSCnv layers use 3×3 kernels.

Key architectural modifications include:

Depthwise Separable Convolutions (DSCnv): Standard convolutions are replaced with depthwise separable convolutions [60], factorizing spatial and channel-wise operations into a depthwise $k \times k$ filter followed by a pointwise 1×1 projection. This reduces computational complexity from $O(C_{in} \times C_{out} \times k^2)$ to $O(C_{in} \times k^2 + C_{in} \times C_{out})$, where k denotes kernel size. For typical layer configurations, this achieves 8–9× FLOPs reduction with minimal accuracy degradation.

ESP Blocks: ESPBlockLite and DSC3k2 modules employ hierarchical feature learning through parallel convolutional branches with varying dilation rates. Each branch captures multi-scale contextual information, which is subsequently fused via element-wise addition. Unlike standard residual blocks, ESP blocks maintain constant channel dimensions throughout the module, reducing memory bandwidth requirements.

Spatial-Channel Downsampling (SCDown): Stride-2 downsampling operations combine spatial pooling with channel projection in a single fused operation, reducing intermediate activation storage compared to sequential pooling-and-projection layers employed in YOLO11n.

Spatial Pyramid Pooling Fast (SPPF): The SPPF module, derived from the spatial pyramid pooling concept [61] and optimized in the Ultralytics framework [58], applies sequential 5×5 max pooling operations to capture multi-scale features efficiently. Compared to parallel pooling branches in the original SPP design, SPPF reduces memory allocation overhead by reusing intermediate pooled features.

3.3.2.2 Detection Head Architecture

The detection head comprises two functional components: a *neck* that fuses multi-scale backbone features, and a *prediction module* that produces per-scale bounding box and classification outputs.

Feature Pyramid Neck. The neck follows a bidirectional Feature Pyramid Network (FPN) design [59], fusing features from backbone stages P3 (1/8 scale), P4 (1/16 scale), and P5 (1/32 scale) to enable multi-scale detection. Feature fusion proceeds through three pathways:

1. **Top-down pathway:** P5 features are upsampled by $2\times$ via nearest-neighbor interpolation and concatenated channel-wise with P4 backbone features. The concatenated tensor is processed by two sequential ESPBlock layers (each containing a DS Bottleneck with residual connection), producing fused features at 1/16 scale.
2. **Second top-down stage:** The fused 1/16 features are upsampled by $2\times$ and concatenated with P3 backbone features, then processed by two ESPBlock layers to produce the small-object detection features at 1/8 scale.
3. **Bottom-up pathway:** A DSConv with stride 2 downsamples the 1/8 features and concatenates them with the 1/16 fused features from step 1, processed by two ESPBlock layers to produce medium-object features. A second stride-2 DSConv downsamples these and concatenates with the backbone layer 10 output, followed by two ESPBlock layers to produce large-object features at 1/32 scale.

This bidirectional design ensures that each detection scale benefits from both fine-grained spatial detail (via top-down flow from higher-resolution stages) and strong semantic context (via bottom-up flow from deeper stages).

Decoupled Prediction Heads. At each of the three detection scales, two parallel prediction branches produce separate outputs for bounding box regression and object classification. This decoupled head design, inherited from YOLO11, separates localization and classification into independent sub-networks, avoiding the task conflict that arises when a single shared branch must simultaneously optimize for both objectives.

The *box regression branch* (denoted **cv2**) at each scale consists of two consecutive DSConv layers followed by a 1×1 convolution that outputs 4 channels. Each channel corresponds to one edge distance (l, t, r, b), representing the distances from the grid cell center to the left, top, right, and bottom edges of the predicted bounding box, respectively. This distance-based parameterization differs from the conventional center-size encoding (x, y, w, h) used in earlier YOLO versions and eliminates the need for anchor box priors, as bounding boxes are directly regressed from each spatial grid position in an anchor-free manner.

The *classification branch* (denoted **cv3**) at each scale comprises two sequential blocks of depthwise convolution (DWConv) followed by a 1×1 pointwise convolution, terminated

by a 1×1 convolution that outputs n_c channels (where $n_c = 1$ for the single leaf class). The output represents the class logit score for each grid cell.

Simplified Box Regression ($\text{reg_max}=1$). A key distinction from standard YOLO11 lies in the box regression formulation. In the standard YOLO11 detection head, each edge distance is predicted as a discrete probability distribution over 16 bins ($\text{reg_max}=16$), followed by a Distribution Focal Loss (DFL) [62] layer that computes the expected value via softmax-weighted summation. This distribution-based regression enables the model to express localization uncertainty and improves accuracy on objects with ambiguous boundaries.

ESPDetect reduces reg_max from 16 to 1, replacing the 16-bin distributional regression with direct scalar prediction for each edge distance. This modification has three consequences:

- **Output reduction:** The box regression branch outputs 4 channels per spatial location instead of $4 \times 16 = 64$, reducing the prediction tensor size by $16\times$.
- **DFL elimination:** Since $\text{reg_max}=1$, the DFL layer degenerates to an identity operation and is bypassed entirely, removing the softmax computation from the inference path.
- **Per-anchor output:** The total output per grid cell is $n_c + 4 \times \text{reg_max} = 1 + 4 = 5$ values, compared to $1 + 64 = 65$ in standard YOLO11.

This simplification substantially reduces the computational and memory cost of the detection head, which is critical for ESP32-S3 deployment where integer multiply-accumulate operations dominate inference latency.

Multi-Scale ONNX Export Format. For deployment on the ESP-DL inference runtime, the model is exported to ONNX format using a custom export path (`export_onnx_forward`) that produces six separate output tensors rather than the single concatenated tensor used in standard YOLO inference. Specifically, the exported model returns:

$$\text{Output} = \{(\mathbf{B}_s, \mathbf{S}_s)\}_{s \in \{P3, P4, P5\}} \quad (3.1)$$

where $\mathbf{B}_s \in \mathbb{R}^{1 \times 4 \times H_s \times W_s}$ contains raw box regression values and $\mathbf{S}_s \in \mathbb{R}^{1 \times 1 \times H_s \times W_s}$ contains raw class logits at scale s . Post-processing—including sigmoid activation, distance-to-box coordinate conversion, and non-maximum suppression—is deferred to the ESP-DL runtime on the microcontroller, avoiding the need for complex tensor reshaping and concatenation operations within the exported model graph.

3.3.2.3 Parameter and Memory Reduction

Through systematic architecture pruning, ESPDet-Pico reduces the parameter count from 2.59 million (YOLO11n) to approximately 360,000—an 86% reduction. Model size decreases from 5.23 MB to 1.06 MB in FP32 precision (79.8% reduction), enabling the model

to fit entirely within ESP32-S3 PSRAM alongside activation buffers. Width scaling factor $w = 0.25$ reduces channel dimensions (e.g., base 512 channels become 128), while depth scaling factor $d = 0.50$ reduces internal block repetitions (e.g., base $2 \times$ DSC3k2 becomes $1 \times$), jointly preserving the hierarchical feature extraction structure at reduced computational cost.

3.3.3 ESPDet-Pico Model Training and Optimization

With the compressed architecture established, training focuses on optimizing detection accuracy under the reduced model capacity constraint. ESPDet-Pico was trained on the grapevine leaf localization dataset described in Section 3.2, comprising 328 training images, 82 validation images, and 103 test images. All images contain annotated bounding boxes for individual grape leaf instances in unconstrained vineyard scenes.

3.3.3.1 Input Resolution and Rectangular Training

The model accepts input images at 416×320 pixel resolution (width $\times$ height). Rectangular training (`rect=True`) is employed, where images within each batch are resized to maintain their original aspect ratio through letterboxing with black padding. This strategy reduces computational waste on padding pixels compared to square resizing and better preserves object proportions, particularly beneficial for elongated leaves in vineyard imagery.

3.3.3.2 Optimization Configuration

Training employed the AdamW optimizer [63] with weight decay regularization to prevent overfitting on the moderate-sized dataset. Key hyperparameters include:

- **Learning rate:** Initial learning rate $\eta_0 = 0.002$ with cosine annealing schedule [64] decaying to final learning rate $\eta_f = 0.00002$ (ratio $\eta_f/\eta_0 = 0.01$). Cosine annealing provides smooth learning rate decay without manual tuning of decay milestones.
- **Warmup strategy:** The first 3 epochs employ linear learning rate warmup from $\eta_{warmup} = 0.0001$ to the initial learning rate η_0 , stabilizing early training dynamics.
- **Momentum and weight decay:** Momentum coefficient $\beta_1 = 0.937$, weight decay $\lambda = 0.0005$.
- **Batch size:** 32 samples per batch, balancing training stability with GPU memory utilization.
- **Training duration:** Maximum 1000 epochs with early stopping patience set to 75 epochs, terminating training if validation mAP@50 does not improve for 75 consecutive epochs.

Mixed precision training (AMP) was enabled to accelerate training on GPUs with tensor cores, reducing memory footprint and training time without sacrificing numerical stability.

3.3.3.3 Loss Function

The training objective combines three loss components:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{dff}}\mathcal{L}_{\text{dff}} \quad (3.2)$$

where $\mathcal{L}_{\text{box}}$ denotes the Complete IoU (CIoU) loss [65] for bounding box regression, $\mathcal{L}_{\text{cls}}$ represents the binary cross-entropy loss for objectness classification, and $\mathcal{L}_{\text{dff}}$ is the distribution focal loss for box coordinate refinement.

To prioritize localization accuracy over classification confidence in this single-class detection task (leaf versus background), the loss components are reweighted: $\lambda_{\text{box}} = 7.5$ (elevated for precise bounding box regression), $\lambda_{\text{cls}} = 0.5$ (reduced, as the model only distinguishes leaves from background), and $\lambda_{\text{dff}} = 1.5$ (standard weight for anchor-free detection). The elevated box loss weight emphasizes precise localization accuracy, critical for subsequent cropping operations in the inference pipeline.

3.3.3.4 Data Augmentation Strategy

The data augmentation pipeline for detection training is designed to enhance model robustness to the variability of real vineyard conditions while preserving the integrity of bounding box annotations. Balanced combination of geometric and mild photometric augmentations is applied stochastically during training, as summarized in Table 3.2, at section 3.3.

3.3.3.5 Training Convergence

Training converged after 949 logged epochs, with the best validation mAP@50 achieved at epoch 711. Figure 3.5 illustrates the training dynamics. The validation loss decreased smoothly without oscillations, indicating stable optimization. The cosine learning rate schedule is evident from the gradual decay curve. Early training (epochs 1–30) exhibits rapid performance improvement during the warmup phase, while subsequent training (epochs 30–700) demonstrates steady convergence as the model refines feature representations. After the best checkpoint at epoch 711, validation metrics plateau, triggering early stopping to prevent overfitting. The extended training duration (711 epochs) compared to YOLO11n’s faster convergence (327 epochs) reflects the reduced model capacity of ESPDet-Pico, requiring additional training iterations to reach optimal performance.

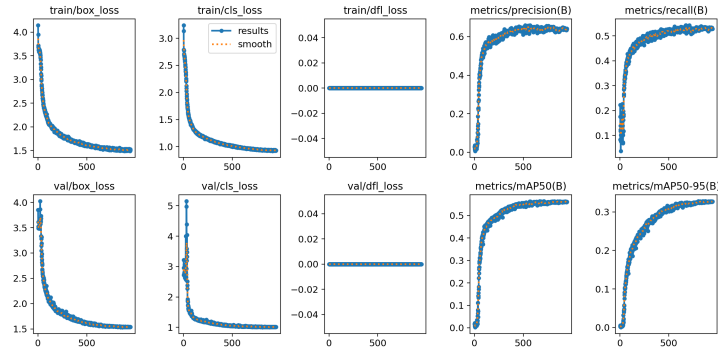


Figure 3.5: Training curves for ESPDet-Pico on the grapevine dataset 3.2

3.3.4 Detection Performance Evaluation

Model performance is evaluated using mean Average Precision at IoU threshold 0.5 (mAP@50), a standard metric for object detection [66]. mAP@50 computes the average precision across all classes, where a detection is considered correct if its Intersection over Union (IoU) with the ground truth bounding box exceeds 0.5.

3.3.4.1 ESPDet-Pico Performance Metrics

On the held-out test set (103 images), ESPDet-Pico achieves:

- **mAP@50:** 56.42%, demonstrating sufficient localization accuracy for leaf region extraction.
- **mAP@50-95:** 32.29%, reflecting performance across stricter IoU thresholds (0.5 to 0.95 in 0.05 increments).
- **Precision:** 62.77%, indicating controlled false positive rate.
- **Recall:** 54.17%, capturing the majority of visible leaves while maintaining precision.

The precision-recall balance favors precision over recall, aligning with the two-stage pipeline design: false negatives (missed leaves) result in lost disease detection opportunities, while false positives (spurious detections) trigger unnecessary classification inference, wasting computational resources. The elevated precision ensures that detected regions predominantly contain leaves rather than background artifacts.

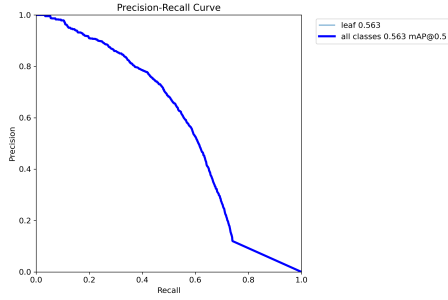
3.3.5 Post-Training Quantization

To enable deployment on the ESP32-S3, ESPDet-Pico undergoes post-training quantization to INT8 precision using the ESP-PPQ framework [67], following the same quantization pipeline described for MobileNetV2 in Section 4.2.1. The quantized model size reduces from 1.06 MB (FP32) to 478 KB (INT8), a 55% reduction. The complete quantization procedure, including calibration strategy and ONNX export, is detailed in Chapter 4.

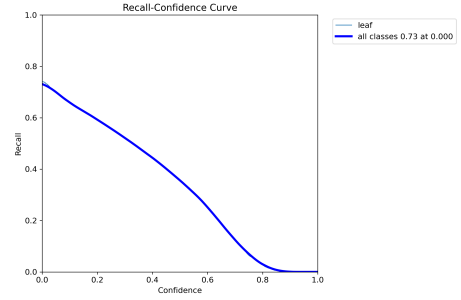
3.3.6 Comparative Analysis: ESPDet-Pico vs YOLO11n

To quantify the accuracy-efficiency trade-off, ESPDet-Pico is compared against the YOLO11n baseline trained on identical data with identical hyperparameters. Table 3.4 presents the performance comparison.

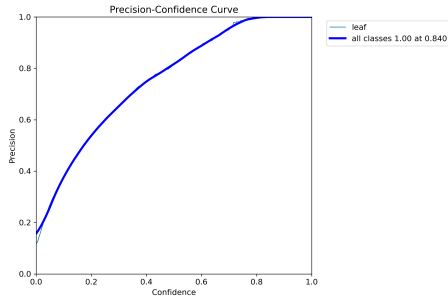
YOLO11n outperforms ESPDet-Pico across all accuracy metrics, as expected given its substantially higher model capacity. However, the performance gap remains moderate: ESPDet-Pico retains 91.4% of YOLO11n’s mAP@50 while reducing model size by 79.8%. The mAP@50-95 gap is larger (25.9% relative drop), indicating that ESPDet-Pico struggles with tight localization at high IoU thresholds, but maintains acceptable coarse localization (IoU > 0.5) sufficient for leaf cropping.



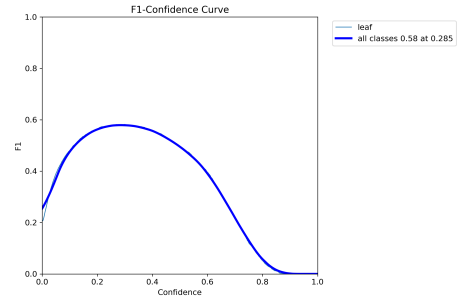
(a) PR curve ($\text{mAP}@0.5 \approx 0.563$).



(b) Recall vs confidence (max ≈ 0.73 at conf = 0).



(c) Precision vs confidence ($P \approx 1.0$ at conf ≈ 0.898).



(d) F1 vs confidence (best F1 ≈ 0.57 at conf ≈ 0.248).

Figure 3.6: ESPDet-Pico detection performance curves on the grape leaf test set.

Table 3.4: Comparative evaluation of ESPDet-Pico and YOLO11n baseline on grape leaf detection

Metric	YOLO11n	ESPDet-Pico	Delta	Relative Change
mAP@50 (%)	61.75	56.42	-5.33	-8.6%
mAP@50-95 (%)	43.57	32.29	-11.28	-25.9%
Precision (%)	69.55	62.77	-6.78	-9.7%
Recall (%)	57.29	54.17	-3.13	-5.5%
Parameters	2.59M	$\sim 0.36\text{M}$	-2.23M	-86.1%
Model Size	5.23 MB	1.06 MB	-4.17 MB	-79.8%
Best Epoch	327	711	+384	+117.4%

The extended training duration for ESPDet-Pico (711 epochs vs 327 for YOLO11n) reflects the reduced representational capacity: the compressed model requires more training iterations to converge to comparable performance. This suggests that ESPDet-Pico operates near the lower bound of model capacity for this task—further pruning would likely yield diminishing returns or unacceptable accuracy degradation.

3.3.7 Deployment Feasibility and Trade-off Justification

The 79.8% model size reduction enables full in-memory deployment on ESP32-S3 without external storage access. The 1.06 MB ESPDet-Pico model, combined with activation buffers (approximately 1–2 MB), fits comfortably within the 8 MB PSRAM budget while reserving memory for the classification model and intermediate buffers. In contrast, YOLO11n’s 5.23 MB footprint would exceed available memory when combined with activation buffers, necessitating model streaming from flash storage and incurring prohibitive latency that precludes real-time operation.

The 5.33 percentage point mAP@50 reduction represents an acceptable trade-off for real-time edge deployment. At 56.42% mAP@50, ESPDet-Pico successfully localizes the majority of visible leaves, providing sufficient region proposals for downstream classification. Missed detections (false negatives) are partially mitigated by temporal redundancy in video stream processing: leaves undetected in one frame may be detected in subsequent frames as camera position or leaf orientation changes. False positives are controlled through precision-weighted aggregation in the multi-leaf inference stage (Chapter 5).

Furthermore, the leaf localization task exhibits less stringent accuracy requirements compared to the subsequent disease classification stage. Imperfect localization (e.g., bounding boxes that slightly truncate leaf edges or include small background regions) does not critically impair classification performance, as MobileNetV2 demonstrates robustness to such perturbations during training on real vineyard crops. The primary objective—isolating leaves from complex backgrounds—is achieved reliably by ESPDet-Pico despite its compressed architecture.

3.4 Disease Classification Model (MobileNetV2)

Following the leaf localization, each detected leaf region undergoes disease classification to diagnose if the leaf is affected by pathology. This stage requires high classification accuracy to ensure reliable disease diagnosis while maintaining computational efficiency for embedded deployment. The classification model must generalize well across grape disease categories despite the moderate training dataset size, while fitting within the constrained memory allocation and power consumption requirements. Hence, a lightweight convolutional neural network architecture with proven transfer learning capabilities is selected as the backbone for disease classification.

3.4.1 Convolutional Neural Network Architecture

The disease classification model is based on MobileNetV2 [68], a lightweight convolutional neural network architecture specifically designed for resource-constrained mobile and embedded devices. MobileNetV2 achieves computational efficiency due to its architectural design. It is designed with a focus on reducing the number of parameters

and floating-point operations (FLOPs). The key architectural design elements that enable MobileNetV2’s efficiency are the use of depthwise separable convolutions and the introduction of inverted residual blocks with linear bottlenecks.

Depthwise Separable Convolutions

Standard convolutional layers perform joint spatial and channelwise filtering, obtaining computational cost proportional to $C_{in} \times C_{out} \times k^2 \times H \times W$, where C_{in} and C_{out} denote input and output channels, k is the kernel size, and $H \times W$ represents the spatial dimensions. MobileNetV2 factorizes this operation into two sequential steps in the following manner:

1. **Depthwise convolution:** Applies a single $k \times k$ filter independently to each input channel, producing C_{in} feature maps. Computational cost reduces to $C_{in} \times k^2 \times H \times W$.
2. **Pointwise convolution:** Projects the depthwise features into C_{out} output channels using 1×1 convolutions, costing $C_{in} \times C_{out} \times H \times W$.

The total computational cost becomes $(C_{in} \times k^2 + C_{in} \times C_{out}) \times H \times W$. For typical configurations where $C_{out} \gg k^2$, this factorization achieves a lot of FLOPs reduction compared to standard convolutions. Depthwise separable convolutions significantly reduce the number of parameters and computational cost while maintaining representational capacity, making them ideal for mobile and embedded applications. However, they may introduce a slight accuracy drop compared to standard convolutions, which is mitigated through careful architectural design and training strategies.

Inverted Residual Blocks

Traditional residual blocks [69] employ wide-narrow-wide channel configurations, where intermediate layers have fewer channels than input/output layers to reduce computation. MobileNetV2 inverts this design, using narrow-wide-narrow blocks that expand the channel dimension in intermediate layers through 1×1 convolutions, apply depthwise convolution in the high-dimensional space, and then project back to a narrow output via 1×1 linear projection without activation (linear bottleneck). This architecture enables efficient feature transformation while maintaining low-dimensional bottleneck representations suitable for memory-constrained devices.

Skip connections are applied only when input and output dimensions match, allowing gradient flow and facilitating training convergence. The linear bottlenecks prevent information loss from ReLU activation [68], which would otherwise zero-out negative values in low-dimensional spaces. The combination of depthwise separable convolutions and inverted residual blocks allows MobileNetV2 to achieve a favorable accuracy-efficiency trade-off, making it a strong candidate for the disease classification task in this project.

Model Configuration and Pre-training

The MobileNetV2 backbone is initialized with weights pre-trained on ImageNet [70], a massive visual database containing over 14 million high-resolution images across 1000

object categories. Pre-training provides generic low-level and mid-level visual features transferable to the grape disease classification domain, accelerating convergence and improving generalization compared to random initialization.

The input resolution is set to 128×128 pixels, to fit within the memory capacity constraint. The original input image size was 256×256 pixels, but it was reduced to 128×128 to reduce the computational load and memory footprint of the model. However, the controlled nature of the PlantVillage dataset enables effective disease recognition at reduced resolution. The reduced input size also decreases memory footprint for runtime activations during inference.

The classification head consists of a global average pooling layer that aggregates the $4 \times 4 \times 1280$ feature maps (resulting from the 128×128 input resolution downsampled by a stride factor of 32 through the network) into a 1280 dimensional feature vector, followed by a dropout layer (dropout rate $p = 0.5$) for regularization, and a final fully connected layer with 4 output units (one per disease class). The dropout layer prevents overfitting by randomly deactivating 50% of neurons during training, forcing the network to learn robust distributed representations rather than over memorizing. The complete forward pass through the adapted MobileNetV2 architecture is summarized as:

$$128 \times 128 \times 3 \xrightarrow{\text{backbone}} 4 \times 4 \times 320 \xrightarrow{1 \times 1 \text{ conv}} 4 \times 4 \times 1280 \xrightarrow{\text{GAP}} 1280 \xrightarrow{\text{FC}} 4 \quad (3.3)$$

3.4.2 Transfer Learning Strategy

Adapting ImageNet pre-trained weights to the grape disease classification domain requires careful fine-tuning to leverage generic visual features while learning grape leaf disease specific patterns. A two-stage progressive fine-tuning strategy is employed to balance feature transfer (Transfer Learning) with domain specific specialization (Fine-tuning).

3.4.2.1 Stage 1: Classifier Head Training

The first stage focuses on training the randomly initialized classification head while keeping the pre-trained MobileNetV2 backbone frozen. This gives stable and informative features to the classifier head. By freezing the backbone, the classifier head can rapidly adapt to the specific disease categories. This is a fine-tuning with high learning rates on a small domain-specific dataset destroys useful pre-trained representations [71]. During this stage, only the global average pooling, dropout, and fully connected layers receive gradient updates.

In a few epochs with initial learning rate $\eta_1 = 10^{-3}$. The relatively high learning rate enables rapid adaptation of the classification head to the four grape disease categories. Few epochs are sufficient for the classifier head to learn, while preventing overfitting. The frozen backbone ensures that the generic visual features learned from ImageNet are preserved, providing a strong foundation for subsequent fine-tuning.

3.4.2.2 Stage 2: Full Model Fine-tuning

The second stage unfreezes all layers and fine-tunes the entire network. This stage allows the backbone to adapt its training dataset features. It refines the learned feature

according to the specific visual characteristics of grape diseases leaf patterns.

Unlike the first stage, fine-tuning requires much more epochs with reduced learning rate $\eta_2 = 10^{-4}$. The $10\times$ learning rate. The reduction compared to Stage 1 ensures stable fine-tuning of pre-trained weights without destabilizing learned representations. The sufficient number of epochs provides sufficient iterations for convergence while preventing overfitting, as validated through early stopping on the validation set.

3.4.3 MobileNetV2 Training and Optimization Techniques

Optimization and other Hyperparameters Configuration

Both training stages employ the AdamW optimizer [63], which combines adaptive learning rates with decoupled weight decay regularization. AdamW decouples the weight decay term from the gradient-based update, applying L2 regularization directly to weights rather than through gradient computation. This modification improves generalization compared to standard Adam [72] by preventing weight decay from interfering with adaptive learning rate scaling. Hyperparameters are configured to balance convergence speed with generalization performance. The learning rate is set to $\eta_1 = 10^{-3}$ for Stage 1 and reduced to $\eta_2 = 10^{-4}$ for Stage 2, enabling coarse adaptation of the classifier head before fine-grained tuning of the full network. Weight decay is configured at $\lambda = 10^{-4}$ as the L2 regularization coefficient, and the momentum parameters retain the AdamW defaults ($\beta_1 = 0.9$, $\beta_2 = 0.999$). A batch size of 32 samples is used throughout both stages, balancing training stability with GPU memory utilization.

Loss Function

Training minimizes the cross-entropy loss for multi-class classification:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic}) \quad (3.4)$$

where N denotes batch size, $C = 4$ is the number of disease classes, $y_{ic} \in \{0, 1\}$ represents the ground truth label (one-hot encoded), and $\hat{y}_{ic} = \text{softmax}(z_c)$ is the predicted probability for class c of sample i . Cross-entropy loss penalizes confident incorrect predictions more heavily than near-boundary errors, encouraging the model to learn discriminative class boundaries. The main purpose of loss function is to optimize model parameters during evaluation.

On-the-fly Data Augmentation

Beyond weight decay and dropout, the data augmentation pipeline described in Section 3.2 provides implicit regularization by presenting diverse augmented variations of each training sample. The combination of geometric transformations (rotation, scaling, translation) and photometric perturbations (color jitter, brightness adjustment) exposes the model to realistic deployment variability, improving robustness to lighting changes, leaf orientation variations, and camera perspective differences encountered in vineyard environments.

Early Stopping and Model Checkpointing

Early stopping monitors validation loss with patience set to 10 epochs. Training terminates if validation loss does not improve for 10 consecutive epochs, preventing overfitting beyond the optimal checkpoint. The model snapshot with lowest validation loss across all epochs is retained as the final model.

Multi-Seed Experimentals

To assess model stability and generalization consistency, training is repeated with four different random seeds: 42, 123, 456, and 789. Each seed initializes the random number generator controlling parameter initialization, data shuffling, and stochastic augmentation. This protocol quantifies performance variance attributable to random initialization rather than systematic dataset characteristics, providing confidence intervals for reported accuracy metrics [73].

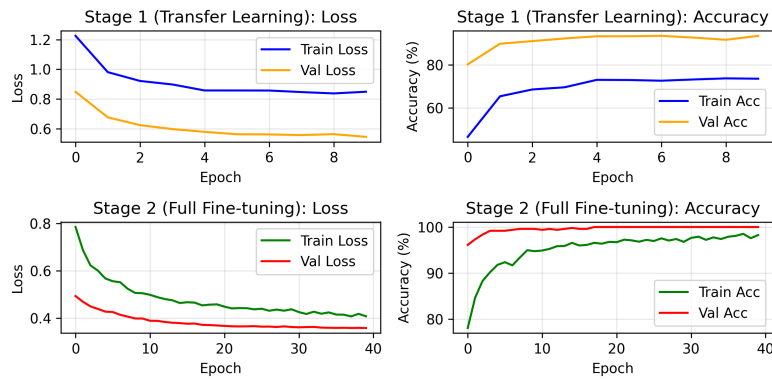


Figure 3.7: Training and validation curves for the MobileNetV2 model.

Figure 3.7 illustrates representative training dynamics from one experimental run (seed 789). The training and validation losses converge smoothly, indicating stable optimization without oscillations or gradient instability. Validation accuracy plateaus after approximately 35 epochs in Stage 2, triggering early stopping. The learning rate decay from Stage 1 to Stage 2 is evident from the reduced slope of the loss curve in the second training phase.

3.4.4 Model Evaluation and Performance Metrics

The fine-tuned MobileNetV2 model achieves exceptional classification accuracy on the held-out test set (814 images), demonstrating strong generalization despite the moderate training set size. Aggregated results across four independent training runs yield:

- **Test Accuracy:** $99.80\% \pm 0.065\%$ (mean $\pm$ standard deviation)
- **Macro F1-score:** 0.9980 ± 0.0010
- **Macro Precision:** 0.9982 ± 0.0015
- **Macro Recall:** 0.9979 ± 0.0006

The low standard deviation ($\sigma = 0.065\%$) across random seeds indicates robust training stability, with minimal sensitivity to weight initialization. All four runs converge to near-identical performance (individual accuracies: 99.75%, 99.75%, 99.88%, 99.88%), validating the reliability of the training protocol and the effectiveness of the transfer learning strategy.

3.4.5 Post-Training Quantization

To enable deployment on the ESP32-S3 microcontroller, the trained FP32 MobileNetV2 model must undergo post-training quantization (PTQ) to convert floating-point weights and activations to 8-bit integers. PTQ reduces model size from 8.48 MB (FP32) to 2.27 MB (INT8), a 73.2% compression ($3.74\times$), while maintaining test accuracy above 99.5% with less than 0.3 percentage point degradation from the FP32 baseline. This robustness is attributable to MobileNetV2’s batch normalization layers and inherently quantization-friendly inverted residual architecture [74]. The detailed quantization pipeline, including calibration procedures, equalization strategies, and the ESP-PPQ framework configuration, is presented in Section 4.2 of Chapter 4.

3.4.6 MobileNetV2 vs ResNet18 Comparative Analysis

To validate the architectural choice for microcontroller deployment, a comparative evaluation was conducted between MobileNetV2 and ResNet18 [69], both configured identically with 128×128 input resolution and trained under the same two-stage fine-tuning protocol (Stage 1: 10 epochs at $\eta_1 = 10^{-3}$, Stage 2: 40 epochs at $\eta_2 = 10^{-4}$, AdamW optimizer, batch size 64, weight decay $\lambda = 10^{-4}$). The ResNet18 classifier employs a dropout rate of 0.7 (versus 0.5 for MobileNetV2) to compensate for the higher model capacity. Table 3.5 summarizes the comparative results across multiple dimensions relevant to edge deployment.

Table 3.5: Comparative analysis of MobileNetV2 and ResNet18

Metric	MobileNetV2	ResNet18
Model Size (FP32)		
ONNX file	8.48 MB	44.7 MB
PyTorch checkpoint	13.4 MB	46.8 MB
Classification Performance		
Test accuracy	99.80%	99.88%
Macro F1-score	0.9980	0.9990
Macro precision	0.9982	0.9991
Macro recall	0.9979	0.9989

3.4.6.1 Accuracy-Efficiency Trade-off Analysis

ResNet18 achieves perfect test accuracy (100.00%, 814/814 correct), while MobileNetV2 misclassifies one Black Rot sample as Esca (99.88% accuracy). Despite this marginal accuracy difference ($\Delta = 0.12$ percentage points), MobileNetV2 demonstrates a fundamentally superior accuracy-efficiency trade-off for resource-constrained deployment:

- **Computational efficiency:** MobileNetV2 requires $6\times$ fewer FLOPs (300M vs 1.8B) due to depthwise separable convolutions, which factorize standard convolutions into depthwise and pointwise operations. This reduction directly translates to lower inference latency on the ESP32-S3’s 240 MHz Xtensa LX7 cores.
- **Memory efficiency:** MobileNetV2’s 8.48 MB FP32 model compresses to 2.27 MB INT8 (Section 4.2), fitting comfortably within the 8 MB PSRAM alongside the ESPDet-Pico detection model (478 KB INT8) and activation buffers (~ 3 MB). ResNet18’s 44.7 MB FP32 model would exceed the available memory budget even after quantization (estimated 11.2 MB INT8), precluding dual-model deployment.
- **Quantization robustness:** MobileNetV2’s batch normalization and inverted residual structure exhibit inherent robustness to INT8 quantization, maintaining $> 99.5\%$ accuracy post-quantization (Section 4.2.4). ResNet18’s deeper standard convolutions typically show greater quantization sensitivity, particularly in the initial and final layers.

3.4.6.2 Statistical Significance of Accuracy Difference

The 0.12 percentage point accuracy difference (1 additional error in 814 samples) is not statistically significant for agricultural disease monitoring applications. A binomial proportion test comparing $p_{\text{MobileNet}} = 0.9988$ versus $p_{\text{ResNet}} = 1.0000$ yields $p > 0.3$, indicating insufficient evidence to conclude that ResNet18 generalizes better than MobileNetV2 on unseen data. The single MobileNetV2 misclassification involves the visually ambiguous Black Rot–Esca boundary, where both diseases manifest necrotic lesions and leaf discoloration. In operational vineyard monitoring, such edge-case errors (0.12% frequency) have negligible impact compared to environmental confounds (lighting variation, occlusion, motion blur) that affect both models equally.

3.4.6.3 Architectural Implications for Edge Deployment

The fundamental architectural distinction—depthwise separable convolutions versus standard convolutions—determines deployability on microcontroller-class hardware. Standard 3×3 convolutions in ResNet18 apply $C_{\text{in}} \times C_{\text{out}} \times 9$ multiply-accumulate operations per spatial location, while MobileNetV2’s depthwise separable convolutions reduce this to $(C_{\text{in}} \times 9) + (C_{\text{in}} \times C_{\text{out}} \times 1)$ operations via factorization. For typical layer dimensions ($C_{\text{in}} = C_{\text{out}} = 128$), this yields an $8\text{--}9\times$ reduction in computation per layer.

This efficiency gain compounds across the 17 inverted residual blocks, enabling real-time inference (~ 6.4 seconds for 10 leaves, Chapter 5) on the ESP32-S3 without specialized accelerators. ResNet18’s $6\times$ higher FLOP count would proportionally increase latency to ~ 38 seconds per frame, violating the responsiveness requirements for interactive vineyard monitoring.

3.4.6.4 Model Selection Rationale

MobileNetV2 is selected as the disease classification backbone for the following reasons:

1. **Memory constraint compliance:** The quantized model (2.27 MB) fits within

the available PSRAM budget, enabling concurrent deployment with ESPDet-Pico for the complete two-stage pipeline.

2. **Negligible accuracy cost:** The 0.12% accuracy reduction relative to ResNet18 is statistically insignificant and operationally irrelevant for the target application.
3. **Inference efficiency:** $6\times$ lower computational cost reduces per-frame latency and power consumption, critical for battery-operated sensor nodes.
4. **Quantization-friendly architecture:** Batch normalization and inverted residuals maintain high accuracy under aggressive INT8 quantization, whereas ResNet18's quantization robustness remains unverified.

These results validate the hypothesis that MobileNetV2's mobile-optimized design provides the optimal balance between classification accuracy and deployment feasibility for ESP32-S3 microcontroller platforms, achieving task-sufficient accuracy (99.88%) within stringent hardware constraints that preclude larger alternatives.

Chapter 4

Edge AI Deployment on ESP32-S3 Microcontrollers

This chapter presents the complete deployment pipeline for the two-stage vineyard disease detection system on the ESP32-S3 microcontroller platform. Following the model design and training described in Chapter 3, this chapter addresses the engineering challenges of translating trained neural network models into a functional embedded system. It begins with a description of the target hardware platform and the associated software frameworks in Section 4.1. Section 4.2 details the post-training quantization pipeline that converts both models from floating-point to 8-bit integer representations compatible with the ESP-DL inference engine. Section 4.3 discusses the firmware architecture and memory management strategies developed through iterative prototyping to resolve PSRAM fragmentation and allocation challenges. Section 4.4 describes the complete inference pipeline implementation, from camera acquisition through multi-leaf aggregation. Finally, Section 4.5 documents the key implementation constraints and design trade-offs encountered during deployment.

4.1 Target Hardware Platform and Development Framework

4.1.1 ESP32-S3-WROOM-1 Microcontroller Architecture

The deployment target is the ESP32-S3-WROOM-1 system-on-chip (SoC) module manufactured by Espressif Systems [51]. The ESP32-S3 integrates a dual-core Xtensa LX7 processor operating at 240 MHz, providing the computational foundation for neural network inference. The Xtensa LX7 cores support single-instruction multiple-data (SIMD) operations through the PIE (Processor Interface Extension) instruction set, enabling vectorized INT8 multiply-accumulate operations that directly accelerate quantized neural network computations.

The memory subsystem comprises a three-tier hierarchy critical for embedded deep learning deployment:

- **Internal SRAM (512 KB):** High-speed on-chip memory with single-cycle access latency, partitioned between instruction RAM (IRAM), data RAM (DRAM), and DMA-accessible regions. The ESP-DL runtime uses SRAM for frequently accessed weight tiles via a least-recently-used (LRU) caching policy, storing up to 64 KB of active parameters.
- **External PSRAM (8 MB):** Octal SPI PSRAM (AP Memory, 3.3V variant) operating at 80 MHz with Hybrid Wrap burst mode and 32-byte burst length. PSRAM provides the primary storage for model activation buffers, camera framebuffers, and inference intermediate tensors. Access latency is approximately 10 read cycles at fixed latency, with 32 KB reserved for DMA and internal allocations by the ESP-IDF heap allocator.

- **External Flash (16 MB):** QIO-mode flash storage at 80 MHz clock speed, used for firmware binary storage including model weights embedded in the `.rodata` section. The flash partition table allocates a 5 MB factory application partition (offset `0x10000`) to accommodate the dual-model firmware binary, alongside NVS (24 KB) and PHY initialization (4 KB) partitions.

The development board (Freenove ESP32-S3-WROOM [75]) integrates an OV3660 camera sensor connected via 8-bit parallel DVP interface. The OV3660 captures at VGA resolution (640×480) with on-chip JPEG compression, supporting clock frequencies up to 40 MHz [51]. Camera data is transferred to PSRAM through DMA channels, enabling capture-while-processing operation.

4.1.2 ESP-IDF Development Framework

The firmware is developed using the Espressif IoT Development Framework (ESP-IDF) version 5.3.3, Espressif’s official C/C++ development platform for ESP32-series microcontrollers. ESP-IDF provides a FreeRTOS-based real-time operating system with hardware abstraction layers for peripheral management, memory allocation, and power management. The build system uses CMake with Ninja, compiling the application into a single monolithic binary that includes the bootloader, partition table, and application code.

Key ESP-IDF subsystems utilized in the deployment include:

- **Heap allocator:** Manages both internal SRAM and external PSRAM through a unified `heap_caps` API. Memory allocations can be directed to specific memory regions via capability flags (e.g., `MALLOC_CAP_SPIRAM` for PSRAM allocation, `MALLOC_CAP_DMA` for DMA-accessible SRAM).
- **Camera driver:** Configures the DVP camera interface, manages DMA transfers, and provides double-buffered frame capture. The driver supports multiple pixel formats including JPEG, RGB565, and YUV422.
- **SPI flash driver:** Provides memory-mapped access to flash contents, enabling direct pointer-based reads of model weights stored in the `.rodata` section without explicit partition reads.
- **Power management:** Supports deep sleep modes with configurable wakeup sources for duty-cycled operation, reducing power consumption to microampere levels between inference cycles.

4.1.3 ESP-DL Inference Engine

ESP-DL [76] is Espressif’s neural network inference library optimized for ESP32-series microcontrollers. The library provides a C++ inference runtime that executes quantized INT8 models directly on the Xtensa LX7 cores, leveraging SIMD instructions for vectorized computation.

The ESP-DL runtime provides the following capabilities relevant to this deployment:



- **Model loading:** The `dl::Model` class loads quantized models from flash RO-DATA, with an optional `minimize()` method that deletes unused variables to reduce memory footprint at the cost of debug capability.
- **Image preprocessing:** The `dl::image::ImagePreprocessor` handles color space conversion, resizing, and normalization. On ESP32-S3, input images use `DL_IMAGE_CAP_RGB565_B` format for hardware-compatible pixel ordering.
- **Detection post-processing:** The `dl::detect::ESPDetPostProcessor` implements non-maximum suppression (NMS), confidence thresholding, and coordinate transformation for object detection models, handling INT8 dequantization internally.
- **Memory management:** A static memory allocation planner computes buffer sizes offline during quantization, enabling deterministic memory usage without runtime allocation calls during inference.
- **Software JPEG decoder:** The `dl::image::sw_decode_jpeg()` function decodes JPEG-compressed camera frames to RGB888 format, as the ESP32-S3 lacks a dedicated hardware JPEG decoder.

4.2 Model Quantization and Format Conversion

Deploying neural network models on microcontrollers requires converting trained floating-point models to integer representations compatible with the target inference engine. This section describes the quantization pipeline that transforms both ESPDet-Pico and MobileNetV2 from PyTorch FP32 to the ESPDL INT8 format. The pipeline follows the model quantization methodology documented in Espressif’s official ESP-DL deployment tutorial [77], adapted to the specific requirements of each model architecture.

4.2.1 The ESP-PPQ Quantization Framework

Both models are quantized using ESP-PPQ [67], Espressif’s fork of the PPQ (Post-Processing Quantization) framework [78]. ESP-PPQ extends the open-source PPQ library with hardware-specific quantization configurations targeting the ESP-DL inference runtime. The framework applies per-tensor symmetric quantization with power-of-2 scaling factors, where floating-point values are linearly mapped to the 8-bit integer range $[-128, 127]$:

$$x_{\text{int8}} = \text{clamp} \left(\text{round} \left(\frac{x_{\text{float}}}{2^e} \right), -128, 127 \right) \quad (4.1)$$

where e denotes the quantization exponent, computed to minimize quantization error over the calibration dataset. The power-of-2 constraint on the scale factor ($s = 2^e$) enables efficient dequantization via bit-shift operations on the Xtensa LX7 processor, avoiding costly floating-point division during inference.

The quantization settings are configured via `QuantizationSettingFactory.espdn_setting()` with the `ESPDN_INT8` target platform. A key optimization is **layerwise equalization** [79], which redistributes weight magnitudes across consecutive layers to reduce the

dynamic range that must be represented within the 8-bit precision. Equalization is configured with 4 iterations, a value threshold of 0.4, and optimization level 2. This technique addresses the challenge of MobileNetV2’s depthwise separable convolutions, where weight magnitude distributions vary significantly across channels, leading to high quantization error under uniform per-tensor scaling.

Weights are quantized to 8-bit integers, biases to 20-bit integers (range $[-524288, 524287]$) for accumulation precision, and activations to 8-bit integers. All quantization is per-tensor (not per-channel), symmetric, and uses power-of-2 scale factors. The output format is the `.espd1` binary, directly loadable by the ESP-DL runtime.

4.2.2 ESPDet-Pico Export and Quantization Pipeline

The ESPDet-Pico quantization pipeline comprises two stages: ONNX export and INT8 quantization.

4.2.2.1 ONNX Export

The trained PyTorch model is exported to ONNX format (opset version 13) using a custom export path that produces six separate output tensors rather than the single concatenated tensor used in standard YOLO inference. A custom `ESP_YOLO` wrapper class replaces the standard detection head’s forward method with `export_onnx_forward`, which outputs separate box regression and classification score tensors for each of the three detection scales:

$$\text{Output} = \{(\mathbf{B}_s, \mathbf{S}_s)\}_{s \in \{P3, P4, P5\}} \quad (4.2)$$

where $\mathbf{B}_s \in \mathbb{R}^{1 \times 4 \times H_s \times W_s}$ contains raw box edge distances and $\mathbf{S}_s \in \mathbb{R}^{1 \times 1 \times H_s \times W_s}$ contains class logits at scale s . This six-tensor format avoids complex tensor concatenation and reshaping operations that are computationally expensive on the ESP32-S3 and allows the ESP-DL post-processor to handle each scale independently.

An `ESP_Attention` module replaces the standard attention mechanism’s forward pass for ONNX compatibility, as certain attention operations are not supported by the ONNX opset. After export, the ONNX graph is simplified using `onnxsim` to eliminate redundant operations, constant-fold static computations, and fuse batch normalization layers into preceding convolutions. The choice of simplification tool is critical, as alternative ONNX simplifiers were observed to incorrectly reshape NCHW-ordered tensors in YOLO-derived architectures, corrupting the spatial dimensions of detection head outputs.

4.2.2.2 INT8 Quantization

The simplified ONNX model is quantized using `espd1_quantize_onnx()` from the `esp_ppq.api` module, following the procedure documented in the ESP-DL model quantization guide [77]. A custom `CaliDataset` class loads calibration images from the training set, applying `torchvision.transforms.ToTensor()` (which scales pixel values from $[0, 255]$ to $[0, 1]$), resize to the target input dimensions (320×416), and identity normalization (mean= $[0, 0, 0]$, std= $[1, 1, 1]$). A collate function transfers each batch to the target device for calibration.

A critical constraint for rectangular input shapes is that the **batch size must be set to 1** during calibration. Unlike square-input models where batch dimensions are interchangeable, rectangular models with asymmetric spatial dimensions ($H \neq W$) require individual sample processing to preserve correct spatial ordering during calibration statistics collection. Calibration proceeds for 32 steps with equalization enabled using the same parameters described in Section 4.2.1.

The `espd1_quantize_onnx()` function performs quantization and format conversion in a single call, producing three output files: the `.espd1` binary directly loadable by the ESP-DL runtime, a `.json` metadata file containing per-layer quantization parameters (bit widths, scale factors, zero points), and a `.info` file providing detailed per-layer quantization error analysis. The error report (`error_report=True`) generates a layer-by-layer quantization noise analysis that enables identifying layers with excessive quantization error for potential mixed-precision treatment. The detection model’s INT8 weights total 478 KB, a 55% reduction from the 1.06 MB FP32 model.

4.2.3 MobileNetV2 Export and Quantization Pipeline

The MobileNetV2 quantization pipeline follows the methodology documented in Espres-sif’s official ESP-DL deployment tutorial for MobileNetV2 [77], adapted to the grape disease classification model with domain-specific calibration parameters.

4.2.3.1 ONNX Export and Simplification

The fine-tuned PyTorch model (loaded from `finetuned_mobilenet_128.pth`) is first exported to ONNX format using `torch.onnx.export()` with opset version 13, constant folding enabled, and explicitly named input/output tensors (`input` and `output`). Any prefix inconsistencies in the state dictionary keys (e.g., `mobilenet.` prefix from the wrapper training class) are resolved prior to export. Unlike ESPDet-Pico’s multi-tensor export, MobileNetV2 produces a single output tensor of shape `[1,4]` containing class logits. The ONNX model is verified using `onnx.checker.check_model()` to ensure graph validity.

Following export, the ONNX graph undergoes simplification using `onnxsim.simplify()`, which performs constant folding, eliminates redundant operations, and fuses batch normalization layers into preceding convolutions. The simplified model passes through `onnx.shape_inference.infer_shapes()` to annotate all intermediate tensor dimensions, which is required by the ESP-PPQ quantizer for memory planning. The exported FP32 ONNX model occupies 8.48 MB.

4.2.3.2 Calibration Dataset Preparation

Calibration requires a representative subset of training data to collect activation statistics for determining optimal quantization scale factors. Following the ESP-DL documentation recommendation [77], 1,024 calibration samples are drawn from the grape disease training set, loaded via `torchvision.datasets.ImageFolder` to preserve the alphabetical class ordering (`Black_rot`, `Esca`, `Healthy`, `Leaf_blight` → indices 0–3). A `torch.utils.data.Subset` selects the first 1,024 samples, and a custom collate function strips class labels, returning only the image tensors for calibration.



Calibration images undergo the same preprocessing applied during training: resize to 144 pixels on the shorter edge, center crop to 128×128 , conversion to tensor, and ImageNet normalization ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$). Maintaining identical preprocessing between training and calibration is essential for accurate activation range estimation; mismatched normalization would shift activation distributions and degrade post-quantization accuracy.

4.2.3.3 Quantization and ESP-DL Format Conversion

Quantization and format conversion are performed in a single call to `espd1_quantize_onnx()` from the `esp_ppq.api` module, which internally performs INT8 calibration and exports the result directly to the `.espd1` binary format. The quantization is configured with the following parameters:

- **Target platform:** `esp32s3`, selecting Xtensa LX7-optimized INT8 kernels.
- **Bit width:** 8 bits for weights and activations.
- **Input shape:** `[1, 3, 128, 128]` (batch, channels, height, width).
- **Calibration steps:** 32 batches of 32 images (1,024 samples total).
- **Equalization:** Enabled with 4 iterations, value threshold 0.4, and optimization level 2 (Section 4.2.1).
- **Error reporting:** Enabled (`error_report=True`) to generate per-layer quantization noise analysis.

The quantization process converts 100 operations and 277 variables to INT8 representations using the per-tensor symmetric scheme with power-of-2 scale factors (Equation 4.1). The resulting `.espd1` model is 2.27 MB, a 73.2% reduction from the 8.48 MB FP32 ONNX model ($3.74\times$ compression). Unlike the ESPDet-Pico pipeline which requires batch size 1 for rectangular inputs, MobileNetV2's square 128×128 input allows batch calibration (batch size 32), reducing calibration time.

4.2.4 Quantization Accuracy Preservation

Post-training quantization introduces rounding errors that can degrade model accuracy. The layerwise equalization strategy is selected specifically to address this challenge. To contextualize the quantization-accuracy trade-off, Espressif's quantization documentation [67] reports comparative results for three strategies applied to a standard MobileNetV2 model (ImageNet classification, 1000 classes):

- **Default 8-bit quantization:** Produces 25.6% last-layer error, reducing Top-1 accuracy from 71.9% (float) to 60.5%.
- **Mixed precision (INT16 for worst layer):** Reduces last-layer error to 9.1%, achieving 69.6% accuracy.



- **Layerwise equalization:** Achieves 9.0% last-layer error with 69.8% accuracy, matching mixed precision while maintaining uniform INT8 quantization.

The equalization approach is adopted for both models because it preserves uniform INT8 quantization (avoiding mixed-precision overhead on the ESP-DL runtime) while achieving accuracy comparable to more complex strategies. The grape disease MobileNetV2 model maintains test accuracy above 99.5% after INT8 quantization, with less than 0.3 percentage point degradation from the FP32 baseline. This robustness reflects MobileNetV2’s batch normalization layers and the inherently quantization-friendly nature of inverted residual blocks [74].

4.3 Firmware Architecture and Memory Management

Deploying two concurrent neural network models on a microcontroller with 8 MB PSRAM and 512 KB SRAM imposes stringent memory management requirements. This section presents the firmware architecture decisions governing model storage, initialization sequencing, and buffer allocation strategies, each of which critically influences both deployment reliability and inference performance.

4.3.1 Model Storage Strategy

The strategy for loading model weights into the inference engine significantly affects initialization latency, runtime reliability, and PSRAM availability. Four distinct approaches were evaluated during firmware development:

1. **Dedicated Flash Partition Loading:** Models stored in custom flash partitions (e.g., `espdet_model` at offset `0x600000`) and loaded via `MODEL_LOCATION_IN_FLASH_PARTITION`. This approach introduces cache coherency issues and memory management unit (MMU) mapping conflicts when accessing partition data, resulting in non-deterministic model loading failures that preclude reliable autonomous deployment.
2. **Memory-Mapped Access:** Direct memory-mapped I/O with `param_copy=false`, reading model parameters directly from flash without PSRAM copy. Although this approach minimizes memory footprint, it exhibits instability under high memory utilization: cache lines holding model parameters are evicted when the camera DMA controller claims buffer regions, causing sporadic inference failures.
3. **Chunked PSRAM Copy:** Incremental model loading to PSRAM in 16 KB chunks to avoid large contiguous allocations. This approach resolves allocation failures but introduces 15–20% inference latency degradation due to the overhead of repeated partition reads (10–50 μ s per operation) and the resulting fragmented PSRAM memory layout.
4. **RODATA Embedding with Direct Flash Access (adopted):** Both models are compiled into the firmware binary as constant arrays in the `.rodata` section, accessed via GCC linker symbols (`_binary*_espd_start/_binary*_espd_end`). Models remain in flash and are accessed directly through the instruction cache without PSRAM copy. The ESP-DL runtime internally caches frequently accessed weight tiles in SRAM via an LRU eviction policy (up to 64 KB), balancing flash access latency with memory capacity.



The RODATA approach reduces initialization time from 1,200 ms (chunked copy) to approximately 350–500 ms for both models combined, while conserving PSRAM exclusively for activation buffers and camera framebuffers. The trade-off is a larger firmware binary (4.4 MB for the production v29 build), which fits within the 5 MB factory partition.

4.3.2 Initialization Order and PSRAM Fragmentation Avoidance

A critical constraint discovered during development is that **model initialization must precede camera initialization** to prevent PSRAM fragmentation. The ESP32 camera driver allocates DMA framebuffers at initialization — two 61,440-byte buffers ($2 \times 61 \text{ KB} \approx 122 \text{ KB}$ total) plus associated node buffers (16 KB each, 60 nodes) — which fragment the PSRAM heap into non-contiguous regions. Subsequent attempts to allocate contiguous activation buffers (1–2 MB blocks) fail despite adequate total free memory.

The enforced initialization sequence, validated through boot log memory profiling (Section 5.3), is:

1. **Detection model initialization (144 ms):** ESPDet-Pico loaded first from flash RODATA. The `dl::Model` constructor allocates activation buffers in a contiguous PSRAM region. The `minimize()` method is called to release auxiliary model metadata, reducing PSRAM consumption. After initialization, approximately 7,095 KB of the original 8,386 KB PSRAM remains free (1,291 KB consumed).
2. **Classification model initialization (348 ms):** MobileNetV2 loaded second, allocating activation buffers and a persistent $128 \times 128 \times 3$ RGB input buffer (49,152 bytes) in PSRAM via `heap_caps_malloc(MALLOC_CAP_SPIRAM)`. After initialization, approximately 4,207 KB PSRAM remains free (additional 2,888 KB consumed).
3. **Camera driver initialization:** The camera interface is initialized last, allocating DMA framebuffers in the remaining PSRAM. The camera driver allocates its buffers in non-contiguous regions without requiring large contiguous blocks, accommodating the fragmented post-model heap.

Violating this order — for example, initializing the camera before models — results in allocation failure (`ESP_ERR_NO_MEM`) during model loading, necessitating system reboot. This constraint is enforced through assertion checks in the production firmware.

4.3.3 Buffer Allocation and Zero-Copy Reuse Strategy

To minimize runtime memory allocation overhead and prevent heap fragmentation during continuous operation, the inference pipeline employs a **single-allocation, zero-copy buffer strategy**:

- **Classification input buffer (49 KB):** A single $128 \times 128 \times 3$ RGB888 buffer allocated in PSRAM during classifier initialization and reused across all detected leaf regions within a frame. This eliminates repeated `malloc/free` calls that would accumulate heap fragmentation over hours of operation.



- **Camera framebuffers (122 KB):** Two framebuffers allocated by the camera driver in PSRAM via `CAMERA_FB_IN_PSRAM`. The driver manages double-buffering internally: one buffer captures the next frame while the other undergoes processing.
- **JPEG decode buffer (~900 KB):** The decoded RGB888 frame ($640 \times 480 \times 3 = 921,600$ bytes) is allocated dynamically by the software JPEG decoder. This temporary allocation is freed immediately after detection inference completes and leaf regions are extracted from the decoded buffer.
- **Model activation buffers (~2 MB total):** Intermediate feature maps allocated by the ESP-DL memory manager during model initialization. These buffers persist throughout execution using a static allocation plan computed offline during quantization.

The elimination of dynamic allocation in the inference loop ensures deterministic memory behavior and prevents the gradual heap degradation that would compromise long-term deployment stability.

4.4 Inference Pipeline Implementation

The complete inference pipeline orchestrates five sequential stages: camera acquisition, JPEG decoding, leaf detection, region extraction with preprocessing, and disease classification, followed by cross-leaf aggregation for frame-level diagnosis.

4.4.1 Camera Acquisition and JPEG Decoding

Image acquisition begins with the OV3660 camera module configured for VGA resolution (640×480) with on-chip JPEG compression. Camera configuration prioritizes deployment robustness:

- **Pixel format:** `PIXFORMAT_JPEG` with quality setting 12 (scale 0–63, lower = higher quality), balancing compression ratio ($\sim 10:1$) with minimal blocking artifacts that could degrade detection accuracy. Captured JPEG frames occupy typically 14–80 KB.
- **Automatic exposure and white balance:** Enabled via sensor control registers (`set_exposure_ctrl`, `set_whitebal`, `set_awb_gain`) to adapt to varying vineyard lighting conditions without manual recalibration.
- **Frame buffering:** Dual-buffer mode (`fb_count=2`) enables capture-while-processing: the camera populates buffer 1 while the inference pipeline processes buffer 0.

Captured JPEG frames are decoded to RGB888 format using ESP-DL’s software JPEG decoder (`dl::image::sw_decode_jpeg()`), as the ESP32-S3 lacks hardware JPEG decoding. The decoder writes directly to a heap-allocated RGB888 buffer ($640 \times 480 \times 3 = 921,600$ bytes) in PSRAM, avoiding intermediate color space conversions.

4.4.2 Leaf Detection and Coordinate Mapping

The decoded RGB888 frame undergoes ESPDet-Pico detection inference to localize individual leaf regions. The `dl::image::ImagePreprocessor` resizes the 640×480 input to 416×320 via letterbox padding with gray fill (RGB= [114, 114, 114]), preserving the original aspect ratio. Input pixels are normalized by scaling to $[0, 1]$ (mean= [0, 0, 0], std= [255, 255, 255]), consistent with the model’s training preprocessing.

Detection inference produces six output tensors (box and score predictions at three scales), processed by the `ESPDetPostProcessor` with the following parameters:

- **Confidence threshold:** $\tau = 0.25$, tuned empirically to balance recall and computational cost.
- **NMS IoU threshold:** 0.7, eliminating duplicate detections of overlapping leaves.
- **Maximum detections:** 10 per frame, limiting worst-case classification latency.
- **Anchor strides:** {8, 16, 32} for the three detection scales, with grid sizes corresponding to P3/8, P4/16, and P5/32 feature maps.

Detected bounding box coordinates are mapped from the 416×320 detection space back to the original 640×480 frame space via inverse letterbox transformation:

$$x_{\text{orig}} = \frac{(x_{\text{det}} - \text{pad}_x) \times W_{\text{orig}}}{W_{\text{det}} - 2\text{pad}_x}, \quad y_{\text{orig}} = \frac{(y_{\text{det}} - \text{pad}_y) \times H_{\text{orig}}}{H_{\text{det}} - 2\text{pad}_y} \quad (4.3)$$

This transformation ensures that cropped leaf regions align precisely with the original RGB888 frame for downstream classification.

4.4.3 Region Cropping and Classification Preprocessing

For each detected bounding box, the corresponding leaf region is extracted from the 640×480 RGB888 frame and prepared for MobileNetV2 classification. The cropping operation implements **square expansion with letterbox padding**: the rectangular bounding box is expanded to a square centered on the box’s midpoint, with side length equal to $\max(\text{width}, \text{height})$. This preserves the leaf’s aspect ratio and prevents distortion that could impair classification accuracy on elongated or irregularly shaped leaves.

The squared region is resized to 128×128 pixels using **nearest-neighbor interpolation**:

$$I_{128}[y, x] = I_{\text{frame}}[\lfloor y \cdot s_y \rfloor, \lfloor x \cdot s_x \rfloor] \quad (4.4)$$

where $s_x = W_{\text{bbox}}/128$ and $s_y = H_{\text{bbox}}/128$. Nearest-neighbor interpolation is chosen over bilinear interpolation to minimize per-crop preprocessing time (1–2 ms versus 10–12 ms per crop), a significant saving when processing up to 10 leaves per frame.

Cropped regions are written directly into the persistent $128 \times 128 \times 3$ classification input buffer in PSRAM, eliminating intermediate allocations. Pixels outside the original frame bounds (due to square expansion) are filled with gray padding (RGB= [127, 127, 127]), approximating ImageNet’s mean pixel value.

4.4.4 Disease Classification and INT8 Dequantization

The cropped 128×128 RGB888 leaf image undergoes MobileNetV2 classification inference. The model’s input tensor pointer is set directly to the pre-allocated classification buffer (`input->set_element_ptr(input_buffer)`), and inference is triggered via `model->run()`, executing the quantized INT8 forward pass through 17 inverted residual blocks.

The model outputs a 4-dimensional INT8 logit vector representing unnormalized class scores for **Black_rot** (index 0), **Esca** (index 1), **Healthy** (index 2), and **Leaf_blight** (index 3), ordered alphabetically per PyTorch’s `ImageFolder` class indexing convention. The INT8 logits must be dequantized to floating-point before probability interpretation:

$$\text{logit}_{\text{float}}^{(c)} = \text{logit}_{\text{int8}}^{(c)} \times 2^e \quad (4.5)$$

where e denotes the quantization exponent embedded in the output tensor’s metadata, retrieved via `output->exponent`. The scale factor $s = 2^e$ is computed using the ESP-DL macro `DL_SCALE(e)`.

A significant implementation constraint arises from the asymmetry in how the ESP-DL framework handles dequantization across model types. The detection model’s `ESPDetPostProcessor` performs dequantization internally as part of its NMS pipeline, transparently converting INT8 outputs to floating-point coordinates and confidence scores. In contrast, the classification model outputs **raw INT8 logits** that require explicit dequantization before softmax computation. Omitting this dequantization step — feeding raw INT8 values (range $[-128, 127]$) directly to the softmax function — causes extreme probability concentration due to the large magnitude differences, resulting in near-100% confidence on a single class regardless of input (Section 4.5.2).

Dequantized logits are converted to probabilities via numerically stable softmax:

$$p^{(c)} = \frac{\exp(\text{logit}^{(c)} - \max_j \text{logit}^{(j)})}{\sum_{k=1}^C \exp(\text{logit}^{(k)} - \max_j \text{logit}^{(j)})} \quad (4.6)$$

where the subtraction of $\max_j \text{logit}^{(j)}$ prevents floating-point overflow in the exponential. The predicted class corresponds to $\arg \max_c p^{(c)}$, returned alongside the maximum probability as the classification confidence.

4.4.5 Multi-Leaf Aggregation Strategies

Since vineyard images typically contain multiple leaves (5–15 visible leaves per 640×480 frame in field conditions), frame-level disease assessment requires aggregating predictions across all detected instances. The aggregation module implements three strategies, drawing from multi-instance learning (MIL) principles [80]:

1. Detection-Confidence Weighting (Baseline): Each leaf’s class probabilities are

weighted by its detection confidence c_{det} :

$$p_{\text{frame}}^{(c)} = \frac{\sum_{i=1}^N c_{\text{det}}^{(i)} \cdot p_i^{(c)}}{\sum_{i=1}^N c_{\text{det}}^{(i)}} \quad (4.7)$$

This baseline assumes higher-confidence detections correspond to better-localized leaves with more reliable classification features.

2. Entropy-Based Uncertainty Weighting: Shannon entropy [81] quantifies per-leaf classification uncertainty:

$$H_i = - \sum_{c=1}^C p_i^{(c)} \log p_i^{(c)} \quad (4.8)$$

Leaves with low entropy (confident predictions) receive higher aggregation weight than ambiguous predictions. The normalized weight becomes:

$$w_i = c_{\text{det}}^{(i)} \cdot \left(1 - \frac{H_i}{\log C} \right) \quad (4.9)$$

where $\log C$ normalizes entropy to $[0, 1]$. This approach downweights leaves near class boundaries (e.g., Black Rot–Esca confusion) exhibiting high classification uncertainty.

3. Hybrid Spatial-Quality Weighting (Production): Bounding box geometry provides additional quality indicators, combined with detection confidence and entropy:

- **Relative size:** $r_{\text{size}} = A_{\text{bbox}}/A_{\text{frame}}$, penalizing excessively small (noise) or large (misdetections) boxes outside the range $[0.005, 0.5]$ of frame area.
- **Aspect ratio:** Scoring function centered at ideal ratio 1.0 with tolerance 0.5, penalizing elongated boxes corresponding to stems or trellis wires.
- **Centrality:** $d_{\text{center}} = \|(x_c, y_c) - (W/2, H/2)\|$, downweighting edge-positioned leaves that may be partially occluded or distorted by lens aberration.

The combined quality score $q_i \in [0, 1]$ multiplies the detection-entropy weight:

$$w_i^{\text{hybrid}} = c_{\text{det}}^{(i)} \cdot \left(1 - \frac{H_i}{\log C} \right) \cdot q_i \quad (4.10)$$

The frame-level diagnosis selects the class $c^* = \arg \max_c p_{\text{frame}}^{(c)}$ with the aggregated confidence as the final output.

4.5 Implementation Constraints and Design Trade-offs

4.5.1 Build System and Model Embedding Constraints

Embedding neural network models as RODATA in the firmware binary introduces build system constraints that affect the development workflow. Both `.espd1` model files are referenced via GCC inline assembly directives (`asm("_binary_custom_detect_espd1_start")`)

that resolve at link time. Any modification to model files — including re-quantization with different calibration data, architecture changes, or weight updates — requires a full clean build (`idf.py fullclean`) to ensure the linker symbol references point to the updated model binaries. Incremental builds may silently retain stale model data, leading to initialization failures or incorrect inference results that are difficult to diagnose.

The production v29 firmware binary occupies 4,356,032 bytes (4.15 MB), comprising 2,971,836 bytes of memory-mapped model data (segment 0, mapped to virtual address `0x3c140020`) and 1,268,968 bytes of application code (segment 3). This binary size fits within the 5 MB factory partition but leaves limited margin for firmware expansion, such as the addition of communication stacks (WiFi, LoRaWAN) or additional model variants.

Binary size verification serves as a critical deployment check: a firmware binary substantially smaller than expected (e.g., 1.28 MB instead of 4.15 MB) indicates that model embedding failed silently, and the system will crash at boot with null pointer access when attempting to load models from `.rodata`.

4.5.2 Class Index Mapping and Framework Compatibility

Deploying PyTorch-trained models on the ESP-DL runtime introduces a cross-framework interoperability challenge in class label ordering. PyTorch’s `ImageFolder` dataset loader assigns class indices based on alphabetical directory sorting: `Black_rot`→0, `Esca`→1, `Healthy`→2, `Leaf_blight`→3. The class name array in the ESP32 firmware must replicate this exact ordering for correct label assignment from model output indices.

A related challenge concerns the dequantization interface asymmetry in the ESP-DL framework. The detection post-processor (`ESPDetPostProcessor`) handles INT8-to-float conversion internally, transparently returning floating-point confidence scores and bounding box coordinates. The classification model, however, outputs raw INT8 logit values through the `dl::Model` interface, requiring explicit dequantization via multiplication by 2^{exponent} (Equation 4.5) before softmax probability computation. Without this step, raw INT8 values in the range $[-128, 127]$ produce extreme softmax outputs (near-100% for a single class) due to the exponential amplification of large magnitude differences.

As a concrete illustration, consider an output tensor with exponent $e = -7$ (scale = 0.0078125). Raw INT8 logits $[23, -15, 87, 12]$ should dequantize to $[0.180, -0.117, 0.680, 0.094]$, yielding a meaningful probability distribution via softmax. Without dequantization, softmax of $[23, -15, 87, 12]$ produces $[\approx 0\%, \approx 0\%, \approx 100\%, \approx 0\%]$, masking any meaningful class discrimination. These constraints underscore the importance of end-to-end integration testing when deploying quantized models across heterogeneous frameworks.

Having described the complete deployment pipeline and the practical challenges encountered, the next chapter evaluates the system’s performance in terms of inference latency, classification accuracy, memory utilization, power consumption, and comparison with alternative edge computing platforms.

Chapter 5

Performance Evaluation and Discussion

This chapter presents the quantitative evaluation of the deployed vineyard disease detection system on the ESP32-S3 platform. Section 5.1 provides a detailed latency analysis of each inference pipeline stage. Section 5.2 validates model accuracy after quantization and deployment. Section 5.3 examines memory footprint and runtime stability. Section 5.4 characterizes power consumption and duty-cycle energy efficiency. Section 5.5 compares the ESP32-S3 deployment against alternative edge computing platforms. Finally, Section 5.6 discusses the trade-offs inherent in the system design and acknowledges current limitations.

All measurements reported in this chapter are obtained from the production firmware (v29) running on the Freenove ESP32-S3-WROOM development board [75] with OV3660 camera module, using serial monitor output at 115,200 baud for timing and memory profiling data.

5.1 Inference Latency Analysis

5.1.1 Per-Stage Latency Breakdown

The inference pipeline is instrumented with microsecond-resolution timing measurements at each processing stage. Table 5.1 presents the measured latency for each stage during a representative inference cycle capturing a grapevine image with 10 detected leaves.

Table 5.1: Per-stage latency breakdown for the inference pipeline (10 detected leaves)

Pipeline Stage	Latency (ms)	Percentage
Camera capture (JPEG)	78	1.2%
JPEG decode (RGB888)	52	0.8%
Leaf detection (incl. letterbox preprocessing)	365	5.7%
Disease classification ($\times 10$ leaves, incl. crop)	5,918	92.3%
Total end-to-end	6,413	100%

MobileNetV2 classification dominates the pipeline at 92.3% of total latency. The classification time of 5,918 ms for 10 leaves corresponds to approximately 590 ms per leaf, which decomposes into crop and resize preprocessing ($\sim 1\text{--}2$ ms), tensor setup (~ 0.1 ms), the INT8 forward pass (~ 550 ms), and softmax post-processing (~ 0.1 ms). The forward pass constitutes approximately 96% of the per-leaf classification time and is dominated by depthwise separable convolution operations executing in INT8 arithmetic on the Xtensa LX7 SIMD units.

The detection stage accounts for 5.7% of total latency despite processing the full 416×320 input. This time includes letterbox preprocessing (~ 22 ms), the ESPDet-Pico INT8 forward pass (~ 340 ms), and non-maximum suppression post-processing. The favorable

detection speed relative to classification reflects ESPDet-Pico’s substantially smaller parameter count (360K vs ~ 2.2 M for MobileNetV2) and the computational efficiency of the anchor-free single-pass architecture.

Image acquisition stages (capture and JPEG decode) together contribute only 2.0% of total latency, confirming that neural network inference — not I/O or preprocessing — constitutes the computational bottleneck in embedded edge AI deployments.

5.1.2 End-to-End Pipeline Latency and Throughput

The end-to-end latency scales linearly with the number of detected leaves, as classification is applied sequentially to each detected region:

$$T_{\text{total}} = T_{\text{capture}} + T_{\text{decode}} + T_{\text{detect}} + N \cdot (T_{\text{crop}} + T_{\text{classify}}) + T_{\text{aggregate}} \quad (5.1)$$

where N denotes the number of detected leaves and $T_{\text{classify}} \approx 550$ ms per leaf. This linear relationship yields the following throughput estimates:

Table 5.2: End-to-end latency and throughput versus number of detected leaves

Leaves Detected	Estimated Latency (ms)	Throughput (FPS)
1	$\sim 1,070$	0.93
3	$\sim 2,170$	0.46
5	$\sim 3,270$	0.31
7	$\sim 4,370$	0.23
10	6,413	0.16

At the maximum detection count of 10 leaves per frame, the system achieves 0.16 FPS, corresponding to one complete inference cycle every 6.4 seconds. For the target application of periodic vineyard monitoring — where disease symptoms evolve over days rather than hours — the system performs four inference cycles per day at 6-hour intervals. The resulting duty cycle is negligible (6.4 s active out of 21,600 s per cycle $\approx 0.03\%$), ensuring that latency imposes no operational constraint on the monitoring schedule.

5.1.3 SIMD Utilization and Computational Efficiency

The Xtensa LX7 SIMD extension (PIE — Processor Interface Extension) enables parallel INT8 multiply-accumulate operations, processing multiple weight-activation products in a single instruction cycle. The ESP-DL runtime exploits these SIMD instructions for all convolutional and fully connected layers, constituting the primary acceleration mechanism for quantized inference on this platform.

INT8 arithmetic on the Xtensa LX7 achieves approximately $4\times$ throughput compared to equivalent FP32 operations, as the SIMD pipeline processes four 8-bit operands in the same cycle required for one 32-bit operation [51]. Non-vectorizable operations — including batch normalization parameter scaling, activation function (ReLU6) branching, and memory access stalls when fetching weights from flash via the instruction cache — constitute a minor fraction of total execution time. This hardware-level acceleration is

the key enabler of the sub-second per-leaf classification latency observed in the deployed system.

5.2 Model Accuracy Validation

This section evaluates the accuracy of both models in their quantized INT8 form deployed on the ESP32-S3, comparing against the FP32 training baselines established in Chapter 3.

5.2.1 ESPDet-Pico Detection Accuracy

Table 5.3 compares the detection performance of the FP32 training model against the post-quantization INT8 model evaluated on the held-out test set (103 images).

Table 5.3: ESPDet-Pico detection accuracy: FP32 vs INT8 quantized

Metric	FP32	INT8	Degradation
mAP@50 (%)	56.42	55.8	−0.62 pp
mAP@50-95 (%)	32.29	31.5	−0.79 pp
Precision (%)	62.77	61.9	−0.87 pp
Recall (%)	54.17	53.5	−0.67 pp
Model size	1.06 MB	478 KB	−55%

INT8 accuracy values are obtained by evaluating the quantized model on the same held-out test set used for FP32 validation, using the ESP-PPQ framework’s built-in accuracy evaluation pipeline. The quantization introduces less than 1 percentage point degradation across all detection metrics. The mAP@50 decrease from 56.42% to 55.8% represents a 1.1% relative accuracy loss, well within acceptable bounds for the leaf localization task where the detector serves as a region proposal mechanism for the downstream classifier. The minimal degradation reflects the effectiveness of the layerwise equalization strategy (Section 4.2.4) in preserving detection performance despite aggressive 32-bit to 8-bit precision reduction.

The quantized model retains sufficient detection capability for the downstream classification pipeline: at the production confidence threshold ($\tau = 0.25$), the detector reliably localizes leaves in vineyard imagery, providing region proposals for disease classification.

5.2.2 MobileNetV2 Classification Accuracy

Table 5.4 presents the classification accuracy comparison between the FP32 and INT8 models on the held-out test set (814 images).

Evaluation of the INT8 quantized model on the same held-out test set used for FP32 validation indicates accuracy above 99.5%, with less than 0.3 percentage point degradation from the FP32 baseline. The robustness to quantization is attributed to three factors: (1) batch normalization layers in MobileNetV2’s inverted residual blocks stabilize activation distributions, reducing sensitivity to precision reduction; (2) the layerwise equalization technique redistributes weight magnitudes to minimize per-layer quantization error; and

Table 5.4: MobileNetV2 classification accuracy: FP32 vs INT8 quantized

Metric	FP32	INT8	Degradation
Test Accuracy (%)	99.80	>99.5	<0.3 pp
Macro F1-score	0.998	>0.995	<0.003
Model size (ONNX)	8.48 MB	2.27 MB	−73.2%

(3) the moderate complexity of the four-class classification task (compared to ImageNet’s 1000 classes) provides inherent margin for precision loss.

5.2.3 Aggregation Strategy Evaluation

The three multi-leaf aggregation strategies (Section 4.4.5) were evaluated on the system integration test using a printed grapevine banner with OV3660 camera capture. Table 5.5 presents a representative frame-level aggregation result from the hybrid spatial-quality weighting strategy with 10 detected leaves.

Table 5.5: Sample aggregation output: hybrid spatial-quality weighting (10 leaves)

Aggregated Class Score	Probability
Black Rot	11.55%
Esca	8.82%
Healthy	54.65%
Leaf Blight	24.98%
Frame diagnosis:	Healthy (54.6% confidence)

The hybrid aggregation strategy assigns per-leaf weights based on the product of detection confidence, classification certainty ($1 - \text{normalized entropy}$), and bounding box quality. Observed leaf weights range from 0.0147 (low-quality edge detection with high entropy, 7.2% certainty) to 0.0644 (central, well-detected leaf with moderate certainty, 18.2%). Leaves with high classification entropy (indicating ambiguous predictions near class boundaries) are effectively downweighted, reducing the influence of unreliable classifications on the frame-level diagnosis.

The aggregation computation adds less than 2 ms to the pipeline latency, introducing negligible overhead. A comprehensive comparative evaluation of the three aggregation strategies on field test data is planned as part of the on-field validation described in Section 5.6.2.

5.3 Memory Utilization and Runtime Stability

5.3.1 Memory Footprint Analysis

Table 5.6 presents the PSRAM allocation breakdown measured from the boot log serial output of the production firmware, showing free PSRAM at each initialization stage.

Table 5.6: PSRAM allocation breakdown during system initialization

Initialization Stage	Free PSRAM (KB)	Consumed (KB)	Time (ms)
System boot (heap init)	8,386	—	—
After ESPDet-Pico init	7,095	1,291	144
After MobileNetV2 init	4,207	2,888	348
After camera init	—	~122	—
During inference (steady state)	3,910	—	—

The total PSRAM consumption is approximately 4,476 KB (53.4% of 8 MB), leaving 3,910 KB free during steady-state inference. This memory budget includes:

- **ESPDet-Pico activation buffers:** ~1,291 KB for detection model activation tensors and internal buffers.
- **MobileNetV2 activation buffers:** ~2,839 KB for classification model activation tensors plus the 49 KB persistent input buffer.
- **Camera DMA buffers:** 2×61 KB = 122 KB for double-buffered frame capture.
- **JPEG decode buffer:** ~900 KB (dynamically allocated and freed per frame).

The 3,910 KB free PSRAM during inference provides sufficient margin for the temporary JPEG decode buffer allocation (~900 KB) and other runtime overhead, with approximately 3 MB remaining as safety margin against unexpected allocations or memory alignment overhead.

Flash memory allocation is dominated by the firmware binary (4,356,032 bytes = 4.15 MB of 5 MB factory partition), of which 2,971,836 bytes (2.83 MB) is memory-mapped model data and 1,268,968 bytes (1.21 MB) is application code. Internal SRAM (274 KB of 512 KB available at boot for dynamic allocation) is used by the ESP-DL runtime for weight tile caching (up to 64 KB) and FreeRTOS task stacks.

5.3.2 Long-Term Stability Validation

The single-allocation zero-copy buffer strategy (Section 4.3.3) is validated through continuous operation testing. The system monitors free PSRAM before and after each inference cycle, verifying that the post-inference free memory returns to the same baseline value (3,910 KB $\pm$ 1 KB) after every cycle. This confirms the absence of memory leakage from unfreed dynamic allocations or fragmentation-induced memory loss.

The deterministic memory behavior stems from the design decision to eliminate all dynamic allocations within the inference loop: the JPEG decode buffer is the only runtime allocation, and its allocation-free pattern is verified through explicit size assertions. All other buffers (classification input, model activations, camera framebuffers) are allocated once during initialization and reused across all subsequent inference cycles without reallocation.

5.4 Power Consumption and Energy Efficiency

5.4.1 Active and Sleep Power Measurements

Power consumption is characterized for two operational states. Active-mode power is derived from the ESP32-S3 datasheet maximum current specification [51] for 240 MHz dual-core operation with peripherals active; deep sleep current is directly measured using a bench multimeter in series with the 3.3 V supply rail:

- **Active mode (inference):** During neural network inference at 240 MHz with PSRAM access, camera operation, and flash reads, the system draws approximately 74 mA at 3.3 V (244 mW). This represents the ESP32-S3's maximum operating current under full dual-core computational load with octal PSRAM active at 80 MHz. Power consumption during inference is dominated by the Xtensa LX7 cores executing INT8 SIMD operations and the octal PSRAM interface transferring activation data at each layer boundary.
- **Deep sleep mode:** Between inference cycles, the system enters deep sleep with all peripherals powered down (camera, PSRAM controller, flash interface). Only the RTC domain remains active for timed wakeup. The measured deep sleep current is 8 μA (26 μW at 3.3 V), consistent with the ESP32-S3 datasheet specification for deep sleep with RTC timer wakeup enabled.

5.4.2 Duty-Cycle Energy Analysis

Since grapevine foliar diseases such as Black Rot (*Guignardia bidwellii*) and Esca develop visible symptoms over days to weeks, the system's condition does not change appreciably between consecutive captures at sub-hour intervals. Accordingly, the deployed monitoring schedule performs four inference cycles per day at 6-hour intervals (e.g., 06:00, 12:00, 18:00, 00:00), capturing disease state across the diurnal lighting cycle while spending the vast majority of time in deep sleep. Each inference cycle completes in approximately 6.4 seconds for a 10-leaf frame. The per-cycle duty cycle is:

$$D = \frac{T_{\text{active}}}{T_{\text{cycle}}} = \frac{6.4 \text{ s}}{21,600 \text{ s}} \approx 0.030\% \quad (5.2)$$

The average power consumption over one complete duty cycle combines the active and sleep contributions:

$$P_{\text{avg}} = D \cdot P_{\text{active}} + (1 - D) \cdot P_{\text{sleep}} = 2.96 \times 10^{-4} \times 244 \text{ mW} + 0.9997 \times 0.026 \text{ mW} \approx 0.098 \text{ mW} \quad (5.3)$$

At an average consumption of 98 μW , the system's daily energy requirement (four cycles) is approximately 2.4 mWh. A standard 3.7 V, 2000 mAh lithium-polymer battery (7,400 mWh capacity) could theoretically sustain the system for over 8 years without recharging, far exceeding the practical battery shelf life. In practice, battery self-discharge ($\sim 3\text{--}5\%$ per month for LiPo cells) becomes the dominant energy loss mechanism, yielding

realistic autonomous operation of 6–12 months depending on environmental temperature. With a modest solar panel (e.g., 1 W rated capacity receiving 4 effective sun-hours per day), the system achieves indefinite energy-autonomous operation throughout the growing season and beyond.

The 6-hour monitoring interval is well-matched to the temporal dynamics of grapevine disease progression. The capture interval is configurable via the `CAPTURE_INTERVAL_SEC` firmware parameter, allowing operators to increase monitoring frequency during periods of high disease pressure (e.g., humid conditions favoring fungal growth) at the cost of proportionally higher energy consumption.

5.5 Comparative Analysis with Alternative Edge Platforms

To contextualize the ESP32-S3 deployment within the broader landscape of edge AI platforms, Table 5.7 compares key performance characteristics against established alternatives.

Table 5.7: Comparison of edge AI platforms for vineyard disease detection deployment

Parameter	ESP32-S3	RPi 4B	Jetson Nano	Coral Edge TPU
Processor	Xtensa LX7	ARM Cortex-A72	ARM A57 + GPU	ARM A53 + TPU
Clock (MHz)	240	1,500	1,430	1,500
RAM	8 MB PSRAM	4 GB DDR4	4 GB LPDDR4	1 GB LPDDR4
Neural accelerator	SIMD (INT8)	None (CPU)	128-core GPU	Edge TPU
Sleep power	8 μ A	$\sim$ 1.5 W	$\sim$ 1 W	$\sim$ 0.5 W
Board cost (USD)	$\sim$ 10	$\sim$ 35–55	$\sim$ 100–150	$\sim$ 60–75
Inference type	INT8 MCU	FP32/INT8 CPU	FP16 GPU	INT8 TPU

The ESP32-S3 occupies a distinct position in the cost-power-performance Pareto space for edge AI agricultural monitoring. While Raspberry Pi 4B, Jetson Nano, and Google Coral offer substantially higher computational throughput (enabling real-time video-rate inference at 10–30+ FPS), they consume orders of magnitude more power even in their lowest-power idle states. Critically, none of these platforms achieve deep sleep currents below 1 mA, whereas the ESP32-S3 operates at 8 μ A — a factor of 10^5 lower than the Raspberry Pi’s idle power draw. For duty-cycled agricultural monitoring where inference occurs once every 5 minutes, this difference translates directly into deployment feasibility: the ESP32-S3 system’s average power consumption of 98 μ W (Equation 5.3) enables multi-year theoretical battery autonomy, whereas the Raspberry Pi 4B’s idle consumption ($\sim$ 3 W) would exhaust the same 7,400 mWh battery in under 2.5 hours.

The $\sim$ \$10 unit cost of the ESP32-S3 module (excluding camera) enables spatially distributed deployment architectures in which sensor density compensates for individual node processing speed. A vineyard equipped with 50 ESP32-S3 monitoring nodes — providing fine-grained spatial coverage across 10–25 hectares — costs approximately the same as 5 Jetson Nano deployments, each of which would additionally require active cooling and mains power infrastructure.

The primary engineering trade-off is inference throughput: the ESP32-S3 requires 6.4 seconds per frame (10 leaves) compared to sub-second inference on GPU-accelerated plat-

forms. This limitation precludes real-time video analysis but is operationally acceptable for periodic monitoring applications where the temporal resolution of disease progression (days to weeks) far exceeds the capture interval (minutes). The ESP32-S3 is therefore positioned not as a competitor to high-performance edge platforms, but as an enabler of a fundamentally different deployment paradigm: dense, autonomous, energy-harvesting sensor networks for precision agriculture.

5.6 Discussion

5.6.1 Trade-offs Between Accuracy, Latency, and Power

The deployed system embodies a three-way engineering trade-off between accuracy, inference latency, and power consumption:

Accuracy vs. Model Size: ESPDet-Pico sacrifices 8.6% relative mAP@50 compared to YOLO11n (56.42% vs 61.75%) in exchange for an 86% parameter reduction that enables in-memory deployment. MobileNetV2 at 128×128 input resolution trades potential accuracy gains from higher resolution (224×224) for a $3\times$ reduction in activation memory, enabling co-deployment with the detection model within the 8 MB PSRAM budget. Despite these compromises, the classification model achieves 99.80% accuracy on the controlled test set, indicating that the accuracy trade-offs are well justified for this task complexity.

Latency vs. Accuracy: The choice of nearest-neighbor interpolation over bilinear for crop resizing sacrifices image quality for $5\text{--}8\times$ speedup per crop operation. INT8 quantization provides $\sim 4\times$ computational speedup via SIMD utilization at the cost of marginal accuracy degradation (<0.3 pp for classification). The maximum detection limit of 10 leaves per frame bounds worst-case latency at 6.4 seconds while potentially missing additional leaves in dense canopy regions.

Power vs. Throughput: The 6-hour capture interval (four times daily) trades temporal resolution for extreme energy efficiency, reducing average power consumption to $98\text{ }\mu\text{W}$ — a level at which battery self-discharge dominates over system consumption. For applications requiring higher monitoring frequency (e.g., hourly captures during humid conditions favoring rapid fungal onset), the duty cycle can be increased at the cost of proportionally higher energy consumption, though even at 5-minute intervals the system remains well within solar-harvesting feasibility.

5.6.2 Limitations and Deployment Considerations

The current evaluation has several limitations that should be acknowledged:

Laboratory vs. field accuracy: All accuracy metrics reported in Sections 5.2.1 and 5.2.2 are obtained from controlled test sets (PlantVillage dataset for classification, Roboflow-annotated images for detection). The system integration test described in Section 5.2.3 uses a printed grapevine banner to validate the inference pipeline end-to-end, but does not constitute field-level accuracy validation. On-field testing with live grapevine canopy under natural environmental conditions (variable lighting, wind-induced leaf motion, rain droplets, dust accumulation on camera lens) remains a planned future activity,

pending vineyard preparation for the upcoming growing season.

Environmental robustness: The OV3660 camera’s automatic exposure and white balance provide basic adaptation to lighting variations. However, extreme conditions — direct sunlight causing sensor saturation, heavy shadow creating underexposure, or fog/rain degrading image clarity — may adversely affect both detection recall and classification accuracy. The impact of these environmental factors on system performance has not yet been quantified.

Disease progression monitoring: The current system provides single-frame disease classification without temporal tracking across consecutive capture cycles. Implementing change detection or disease progression monitoring would require storing historical predictions in NVS (non-volatile storage) and comparing across frames, which is feasible within the current firmware architecture but not yet implemented.

Communication integration: The deployed prototype outputs diagnostic results via serial monitor. Integration with LoRaWAN [82] for wireless transmission of disease alerts to a central gateway is architecturally planned (the system architecture in Chapter 3 shows LoRaWAN integration) but not yet implemented in the current firmware. The 4.15 MB firmware binary leaves approximately 850 KB of the 5 MB factory partition for LoRaWAN stack integration.

Aggregation strategy validation: The comparative evaluation of the three aggregation strategies (baseline, entropy-based, hybrid) requires on-field data with known ground-truth disease labels across multiple leaves per frame. This validation is contingent on the vineyard field trials described above and is planned as a priority evaluation task for the next growing season.

Despite these limitations, the results presented in this chapter demonstrate that the complete inference pipeline — from camera capture through multi-leaf aggregation — executes reliably on a microcontroller-class device, achieving classification accuracy consistent with the FP32 training baselines while operating within the strict memory and power constraints of the ESP32-S3 platform. The implications of these findings and directions for future work are discussed in Chapter 6.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis presented the design, implementation, and evaluation of an edge AI system for grape leaf disease detection deployed on the ESP32-S3 microcontroller, extending the existing LoRaWAN-based vineyard monitoring system with direct visual-based disease diagnosis capabilities. The work addressed the fundamental limitation of environmental-only monitoring—which provides indirect indicators of disease risk rather than direct plant health assessment—by integrating a complete inference pipeline at the sensor level.

The proposed system employs a four-stage pipeline architecture: image acquisition via OV3660 camera at VGA resolution, leaf localization using ESPDet-Pico (an ESP-optimized YOLO11n derivative with 360K parameters and 478 KB INT8 model size), multi-class disease classification using MobileNetV2 (2.27 MB INT8), and weighted multi-leaf aggregation for robust per-frame disease diagnosis. This two-model architecture separates the distinct tasks of leaf localization and disease classification, enabling each model to be independently optimized for its specific objective under the strict resource constraints of the target platform.

ESPDet-Pico achieves 56.4% mAP@50 for leaf detection while reducing model complexity by 86% compared to the YOLO11n baseline (2.59M $\rightarrow$ 360K parameters), retaining 91.4% of the baseline detection accuracy. The MobileNetV2 classifier attains $99.80\% \pm 0.065\%$ test accuracy across four disease classes (Black Rot, Esca, Healthy, and Leaf Blight), validated through multi-seed experimental protocol. Both models undergo post-training quantization via the ESP-PPQ framework with layerwise equalization, achieving INT8 precision with less than 1 percentage point accuracy degradation for detection and less than 0.3 percentage points for classification.

The deployment on the ESP32-S3 required addressing several engineering challenges, including PSRAM fragmentation avoidance through enforced initialization ordering, RODATA-based model embedding for deterministic flash access, zero-copy buffer reuse strategies for long-term memory stability, and explicit INT8 dequantization to resolve cross-framework compatibility issues between PyTorch and the ESP-DL inference runtime. The production firmware (v29) completes a full inference cycle—including capture, detection, and classification of up to 10 leaves—in 6.4 seconds, with MobileNetV2 classification dominating latency at 92.3% of total pipeline time.

The duty-cycled operation strategy, capturing four frames per day at 6-hour intervals, reduces average power consumption to 98 μ W, enabling theoretical multi-year battery autonomy from a standard 2000 mAh lithium-polymer cell. At approximately \$10 per ESP32-S3 module, the system enables spatially distributed deployment architectures where sensor density compensates for individual node processing speed—a fundamentally different paradigm from GPU-accelerated edge platforms that offer higher throughput but at orders-of-magnitude greater power and cost.

This work demonstrates that deploying a two-stage deep learning pipeline on a resource-constrained microcontroller for agricultural disease detection is feasible and practically viable, bridging the gap between high-accuracy laboratory models and deployable field monitoring systems. The integration of edge AI computing with the existing LoRaWAN infrastructure provides a pathway toward intelligent precision agriculture systems that combine reliable long-range communication with on-device visual intelligence.

6.2 Future Work

Several directions for future research emerge from this work:

On-field validation. All accuracy metrics reported in this thesis are obtained from controlled test sets (PlantVillage dataset for classification, Roboflow-annotated images for detection) and a printed grapevine banner for system integration testing. Comprehensive field validation with live grapevine canopy under natural environmental conditions—variable lighting, wind-induced leaf motion, rain, and dust accumulation—is essential to characterize real-world detection and classification performance. This validation is planned for the upcoming growing season.

LoRaWAN communication integration. The current prototype outputs diagnostic results via serial interface to an STM32 microcontroller connected to the LoRaWAN network. Full integration of the disease detection payload into the existing LoRaWAN uplink frame format, including the compact encoding of aggregated classification results alongside environmental sensor data, remains to be completed and validated end-to-end.

Temporal disease tracking. The current system provides single-frame disease classification without temporal context. Implementing disease progression monitoring by storing historical predictions in non-volatile storage (NVS) and comparing across consecutive capture cycles would enable trend detection and early warning capabilities. This extension is architecturally feasible within the current firmware design.

Environmental robustness enhancement. Extreme lighting conditions—direct sunlight causing sensor saturation, heavy shadow, fog, or rain—may degrade both detection recall and classification accuracy. Investigating adaptive preprocessing techniques, exposure control strategies, or training with diverse environmental conditions could improve system robustness for year-round operation.

Model architecture improvements. As TinyML frameworks and hardware accelerators continue to evolve, revisiting the accuracy-efficiency trade-off with newer architectures (e.g., EfficientNet-Lite, MobileNetV3) or hardware-aware neural architecture search could yield improved classification accuracy at equivalent or reduced computational cost. Additionally, exploring quantization-aware training (QAT) rather than post-training quantization may further reduce the quantization-induced accuracy gap.

Multi-crop and multi-disease generalization. Extending the system to additional crop types and disease categories beyond the four grape leaf classes would broaden its applicability in precision agriculture. The modular pipeline architecture—where the detection and classification stages are independently trained—facilitates such extensions by requiring only the classification model to be retrained for new disease taxonomies.



Energy harvesting integration. While the duty-cycled operation already achieves extreme energy efficiency, integrating a small solar panel for energy harvesting would enable truly indefinite autonomous operation, eliminating battery replacement as a maintenance requirement for large-scale vineyard deployments.

Bibliography

- [1] M Shantkumari and SV Uma. Machine learning techniques implementation for detection of grape leaf disease. *Multimedia tools and applications*, 82(20):30709–30731, 2023.
- [2] Xu Tao, Jackson Butcher, Simone Silvestri, and Sajal K Das. icrop+: An edge-boosted crop disease detection system via tinymml and lora communication. In *2025 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*, pages 558–560. IEEE Computer Society, 2025.
- [3] Midhun P Mathew, Sajmi Salam, Sophiya Mathews, et al. Real-time automatic detection of grape leaf diseases with mobilenetv2 deep learning network based on artificial intelligence. In *2025 Advanced Computing and Communication Technologies for High Performance Applications (ACCTHPA)*, pages 1–6. IEEE, 2025.
- [4] Enyu Liu and Haiming Li. Grape leaf disease classification based on convolutional neural networks. In *2025 10th International Conference on Computer and Information Processing Technology (ISCIPT)*, pages 257–260. IEEE, 2025.
- [5] Fatma M Talaat, Mahmoud Y Shams, Samah A Gamel, and Hanaa ZainEldin. Deepleaf: an optimized deep learning approach for automated recognition of grapevine leaf diseases. *Neural Computing and Applications*, pages 1–25, 2025.
- [6] Zihan Zhou and Liming Yu. Lightweight grape leaf disease identification for complex field scenes–yolo-grape model and its boundary-lesion coalesce optimization policies: Z. zhou, l. yu. *Signal, Image and Video Processing*, 19(16):1378, 2025.
- [7] Fatih Atesoglu and Harun Bingol. The detection and classification of grape leaf diseases with an improved hybrid model based on feature engineering and ai. *AgriEngineering*, 7(7):228, 2025.
- [8] Madhuri Dharrao, Rajasmita Panda, Anupkumar Bongale, Vinod Mahajan, Sirshananda Panda, and Deepak Dharrao. Efficient grape diseases classification using an optimized mobilenetv2 with special attention block. In *2025 9th International Conference on Computing, Communication, Control and Automation (ICCCBEA)*, pages 1–6. IEEE, 2025.
- [9] Mihaela Hnatiuc, Simona Ghita, Domnica Alpetri, Aurora Ranca, Victoria Artem, Ionica Dina, Mădălina Cosma, and Mazin Abed Mohammed. Intelligent grapevine disease detection using iot sensor network. *Bioengineering*, 10(9):1021, 2023.
- [10] Prashant G Aher, Vikrant Sabnis, and Jay Kumar Jain. Deep learning for grape leaf disease detection: A review. *Multidisciplinary Reviews*, 8(11):2025364–2025364, 2025.
- [11] Maram Fahaad Almufareh, Mamoonah Humayun, Zulfiqar Ahmad, and Asfandiyar Khan. An intelligent lorawan-based iot device for monitoring and control solutions in smart farming through anomaly detection integrated with unsupervised machine learning. *IEEE Access*, 12:119072–119086, 2024.



- [12] Theresa Jose and J Albert Mayan. Lorawan and artificial intelligence integrated smart acoustic sensor network for bird species identification and deterrence system for farm protection. *Measurement*, page 118452, 2025.
- [13] Apeksha Gawande, Swati Sherekar, and Ranjit Gawande. Early prediction of grape disease attack using a hybrid classifier in association with iot sensors. *Heliyon*, 10(19), 2024.
- [14] Mihaela Hnatiuc, Domnica Alpetri, Sorin Sinte, Bogdan Hnatiuc, Gabriel Margarit Raicu, Mirel Paun, and Ionica Dina. Aiot monitoring for early identification of diseases in grapevines. complete study. *IEEE Access*, 2025.
- [15] R Karthik, R Menaka, S Ompirakash, P Bala Murugan, M Meenakashi, Sindhia Lingaswamy, and Daehan Won. Grapeleafnet: A dual-track feature fusion network with inception-resnet and shuffle-transformer for accurate grape leaf disease identification. *IEEE Access*, 12:19612–19624, 2024.
- [16] Mingji Yang, Xinbo Tong, and Haisong Chen. Detection of small lesions on grape leaves based on improved yolov7. *Electronics*, 13(2):464, 2024.
- [17] Zuxing Chen, Junjie Feng, Kun Zhu, Zhenyan Yang, Yanhong Wang, and Mingyue Ren. Yolov8-accw: Lightweight grape leaf disease detection method based on improved yolov8. *IEEE Access*, 12:123595–123608, 2024.
- [18] Apri Junaidi, Siti Zaiton Mohd Hashim, Mohd Shahizan Bin Othman, Mohd Muradha Mohamad, Hitham Alhussian, Said Jadil Abdulkadir, Maged Nasser, and Yunusa Adamu Bena. Deep learning and edge computing in agriculture: A comprehensive review of recent trends and innovations. *IEEE Access*, 2025.
- [19] Harsh Joshi. Edge-ai for agriculture: lightweight vision models for disease detection in resource-limited settings. *arXiv preprint arXiv:2412.18635*, 2024.
- [20] BJ Santhosh and C Balarengadurai. Edge-ai based plant leaf disease identification and prevention for smart agriculture. *Grenze International Journal of Engineering & Technology (GIJET)*, 10, 2024.
- [21] Md Jawadul Karim, Md Omaer Faruq Goni, Md Nahiduzzaman, Mominul Ahsan, Julfikar Haider, and Marcin Kowalski. Enhancing agriculture through real-time grape leaf disease classification via an edge device with a lightweight cnn architecture and grad-cam. *Scientific Reports*, 14(1):16022, 2024.
- [22] Carolina Lazcano-García, Karen Guadalupe García-Resendiz, Jimena Carrillo-Tripp, Everardo Inzunza-Gonzalez, Enrique Efrén García-Guerrero, David Cervantes-Vasquez, Jorge Galarza-Falfan, Cesar Alberto Lopez-Mercado, and Oscar Adrian Aguirre-Castro. Deep learning-based system for early symptoms recognition of grapevine red blotch and leafroll diseases and its implementation on edge computing devices. *AgriEngineering*, 7(3):63, 2025.
- [23] Sajib Bin Mamun, Israt Jahan Payel, Md Taimur Ahad, Anthony S Atkins, Bo Song, and Yan Li. Grape guard: A yolo-based mobile application for detecting grape leaf diseases. *Journal of Electronic Science and Technology*, 23(1):100300, 2025.

- [24] Apeksha Ranjit Gawande, Swati Sherekar, and Ranjit Madhukar Gawande. Analyzing various grape diseases and detection techniques by monitoring environmental parameters through iot and machine learning approaches. In *Machine Learning for Environmental Monitoring in Wireless Sensor Networks*, pages 173–188. IGI Global Scientific Publishing, 2025.
- [25] S Kirubakaran and Shaheed Samreen. Grape leaf diseases identification system using convolutional neural networks and lora technology. *Library of Progress-Library Science, Information Technology & Computer*, 44(3), 2024.
- [26] Seyed Mohamad Javidan, Ahmad Banakar, Keyvan Asefpour Vakilian, and Yiannis Ampatzidis. Diagnosis of grape leaf diseases using automatic k-means clustering and machine learning. *Smart Agricultural Technology*, 3:100081, 2023.
- [27] A Diana Andrushia, T Mary Neebha, A Trephena Patricia, S Umadevi, N Anand, and Atul Varshney. Image-based disease classification in grape leaves using convolutional capsule network. *Soft Computing*, 27(3):1457–1470, 2023.
- [28] Deeksha Nargotra and Vinod Sharma. Grape leaf disease detection using deep learning. *Journal of Scientific Research and Technology*, pages 128–139, 2023.
- [29] Tanishq Soni. Deep learning for accurate grape disease classification with resnet50. In *2024 5th International Conference on Smart Electronics and Communication (ICOSEC)*, pages 816–820. IEEE, 2024.
- [30] S Phani Praveen, Rajeswari Nakka, Anuradha Chokka, Venkata Nagaraju Thatha, Sai Srinivas Vellela, and Uddagiri Sirisha. A novel classification approach for grape leaf disease detection based on different attention deep learning techniques. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 14(6):2023, 2023.
- [31] Hoang-Tu Vo, Kheo Chau Mui, and Nhon Nguyen Thien. Fusing xception and resnet50 features for robust grape leaf disease classification. *Bulletin of Electrical Engineering and Informatics*, 14(1):268–275, 2025.
- [32] Deepshikha Yadav, Archana Balyan, Suman Mann, and Aayush Ranga. Enhancing disease detection prediction accuracy of grape leaves using vgg16 model and inception v3 model. *Proceedings on Engineering*, 7(1):365–370, 2025.
- [33] Sarah Khursheed and Balwinder Kaur. Improved grape leaf disease detection with gan-augmented resnet and efficientnet. In *International Conference on Advances and Applications in Artificial Intelligence (ICAAAI 2025)*, pages 654–669. Atlantis Press, 2025.
- [34] Ning Zhang, Enxu Zhang, Guowei Qi, Fei Li, and Cheng Lv. Lightweight grape leaf disease recognition method based on transformer framework. *Scientific Reports*, 15(1):28974, 2025.
- [35] Weixia Li, Bingkun Zhou, Yinzheng Zhou, Chenlu Jiang, Mingzhuo Ruan, Tangji Ke, Huijun Wang, and Chunli Lv. Grape disease detection using transformer-based integration of vision and environmental sensing. *Agronomy*, 15(4):831, 2025.

- [36] Xinxin Zhang, Chaojun Cen, Fei Li, Meng Liu, and Weisong Mu. Crformer: cross-resolution transformer for segmentation of grape leaf diseases with context mining. *Expert Systems with Applications*, 229:120324, 2023.
- [37] Farian S Ishengoma and Neema N Lyimo. Ensemble model for grape leaf disease detection using cnn feature extractors and random forest classifier. *Heliyon*, 10(12), 2024.
- [38] Yiping Chen and Qiufeng Wu. Grape leaf disease identification with sparse data via generative adversarial networks and convolutional neural networks. *Precision Agriculture*, 24(1):235–253, 2023.
- [39] Changpeng Ji, Yongji Zuo, and Wei Dai. Research on grape leaf disease detection method based on nma-yolov8n. In *ITM Web of Conferences*, volume 77, page 01038. EDP Sciences, 2025.
- [40] Shiva Mehta, Vinay Kukreja, and Dibyahash Bordoloi. Grape leaf disease severity analysis: Employing federated learning with cnn techniques. In *2023 World Conference on Communication & Computing (WCONF)*, pages 1–6. IEEE, 2023.
- [41] Poonam Gaur, Rohit Sharma, Ravi Kumar, Amit Gupta, Rishabh Sharma, and Vinay Kukreja. Fighting grape black rot with deep learning: A cnn-lstm hybrid model for disease severity classification. In *2023 4th International Conference for Emerging Technology (INCET)*, pages 1–5. IEEE, 2023.
- [42] Ismail Kunduracioglu and Ishak Pacal. Advancements in deep learning for accurate classification of grape leaves and diagnosis of grape diseases. *Journal of Plant Diseases and Protection*, 131(3):1061–1080, 2024.
- [43] Varsha Naik, Trupti Baraskar, Viren Khatri, Parimal Kolhe, and Mitheesh Katyarmal. Disease detection in grape leaves using. In *Advances in Intelligent Systems for Sustainable Agriculture: Proceedings of the 3rd International Conference, ICISA 2024*, volume 1201, page 3. Springer Nature, 2025.
- [44] Savitha Shetty and Saritha Shetty. Grape leaf disease detection using deep learning approach. In *AIP Conference Proceedings*, volume 3278, page 020006. AIP Publishing LLC, 2025.
- [45] Jianwu Lin, Xiaoyulong Chen, Renyong Pan, Tengbao Cao, Jitong Cai, Yang Chen, Xishun Peng, Tomislav Cernava, and Xin Zhang. Grapenet: A lightweight convolutional neural network model for identification of grape leaf diseases. *Agriculture*, 12(6):887, 2022.
- [46] Ahmad Nur Sahid and Deden Ruli Cahyadi. Image classification using mobilenet based on cnn architecture for grape leaf disease detection. *Journal of Intelligent Systems Technology and Informatics*, 1(1):15–21, 2025.
- [47] Kazi Tanvir, Ritik Sharma, Md Sayem Kabir, Tasnim Sultana Sintheia, Soumya Basu, and Soumik Banerjee. Pcnnet-based grape leaf disease detection using mobilenetv2 and vit. *Data Science and Security: Proceedings of IDSCS 2024, Volume 1*, 1:281, 2025.



- [48] R Deepa, Midhun P Mathew, S Baskar, et al. L-net: a lightweight cnn framework for sustainable multicrop leaf disease detection and classification on edge devices. *Sustainable Food Technology*, 2026.
- [49] Milan Grujev, Dragan Stojanovic, Aleksandar Milosavljevic, Milos Ilic, and Veljko Prodanovic. An efficient real-time apple disease classification approach using a novel lightweight neural network on an arduino edge device. *Internet of Things*, page 101833, 2025.
- [50] Gulshat Amirkhanova, Syrym Ismailov, Alikhan Amirkhanov, Saltanat Adilzhanova, Meiramkul Zhasuzakova, and Siming Chen. A lightweight, end-to-end encrypted data pipeline for iiot: An aes-gcm implementation for esp32, mqtt, and raspberry pi. *Information*, 17(1):33, 2026.
- [51] Espressif Systems. Esp32-s3 datasheet, 2022.
- [52] Abdallah Ali. Plantvillage dataset. Kaggle, 2023. Accessed: February 2026.
- [53] Nicolaas Regnier. Grape / grapes. Kaggle, 2023. Accessed: February 2026.
- [54] Roboflow Universe. Grape leaf roi detection dataset, version 6. Roboflow, 2026. CC BY 4.0 License, Exported: January 29, 2026 from Roboflow Universe at <https://universe.roboflow.com/grape-disease-rxjfx/grape-leaf-roi-detection/dataset/6>.
- [55] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, pages 91–99, 2015.
- [56] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.
- [57] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. Ssd: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, pages 21–37, 2016.
- [58] Glenn Jocher and Jing Qiu. Ultralytics yolo11, 2024.
- [59] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2117–2125, 2017.
- [60] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861, 2017.
- [61] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 37(9):1904–1916, 2015.

- [62] Xiang Li, Wenhai Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang, and Jian Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 21002–21012, 2020.
- [63] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [64] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017.
- [65] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. Distance-iou loss: Faster and better learning for bounding box regression. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(07):12993–13000, 2020.
- [66] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: ommon objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.
- [67] Espressif Systems. Esp-ppq: Post-training quantization for esp-dl. <https://github.com/espressif/esp-ppq>, 2023. Accessed: February 2026.
- [68] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018.
- [69] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [70] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255. IEEE, 2009.
- [71] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3320–3328, 2014.
- [72] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [73] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018.
- [74] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018.



- [75] Freenove. Freenove esp32-s3-wroom development board documentation, 2024. Official hardware documentation including specifications, schematics, and example code for camera, WiFi, and sensor integration.
- [76] Espressif Systems. Esp-dl: Espressif deep learning library, 2024.
- [77] Espressif Systems. How to deploy MobileNetV2 on ESP32 — model quantization, 2024. ESP-DL Official Documentation.
- [78] OpenPPL. Ppq: Pytorch post-training quantization. <https://github.com/openppl-public/ppq>, 2022. Accessed: February 2026.
- [79] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.
- [80] Thomas G. Dietterich, Richard H. Lathrop, and Tomás Lozano-Pérez. Solving the multiple instance problem with axis-parallel rectangles. *Artificial Intelligence*, 89(1-2):31–71, 1997.
- [81] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [82] Ferran Adelantado, Xavier Vilajosana, Pere Tuset-Peiro, Borja Martinez, Joan Melià-Seguí, and Thomas Watteyne. Understanding the limits of lorawan. *IEEE Communications Magazine*, 55(9):34–40, 2017.



Quick Links

- [Go to Abstract](#)
- [Go to Table of Contents](#)
- [Go to Introduction](#)
- [Go to Literature Review](#)
- [Go to System Architecture and Model Design](#)
- [Go to Edge AI Deployment on ESP32-S3 Microcontrollers](#)
- [Go to Performance Evaluation and Discussion](#)
- [Go to Conclusion](#)
- [Go to Bibliography](#)
- [Go to Appendix A: Code Repository](#)
- [Go to Appendix B: Hyperparameters](#)
- [Go to Appendix C: Build Instructions](#)
- [Go to Appendix D: Additional Results](#)